

COMPLETE JAVA NOTES

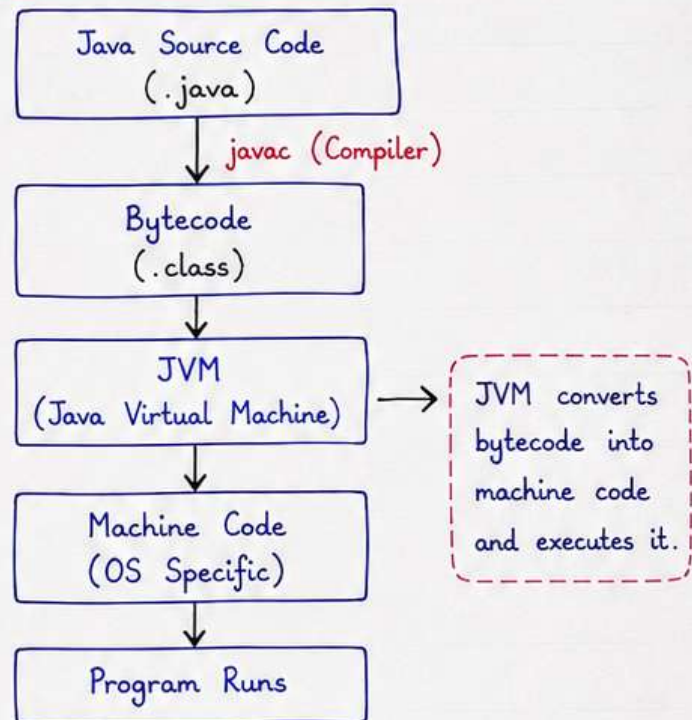
BEGINNER TO ADVANCED

1. What is Java ?

- Java is a high-level, class-based, object-oriented programming language.
- Developed by Sun Microsystems in 1995 and now owned by Oracle.
- Java is platform independent, secure, robust and designed for simplicity.
- Write Once, Run Anywhere (WORA) is the power of Java.

Write Once, Run Anywhere
Compile once and run on any platform that has JVM.

2. How Java Works ?



3. Why Java ?

- Platform Independent (WORA)
- Object Oriented
- Simple and Easy to learn
- Robust and Secure
- Multithreaded
- High Performance (JIT)
- Distributed
- Automatic Memory Management (Garbage Collection)

4. Java Editions

- Java SE (Standard Edition)
Core Java. Used for desktop applications and basic programs.
- Java EE / Jakarta EE (Enterprise Edition)
Used for building enterprise applications like web apps, APIs, etc.
- Java ME (Micro Edition)
Used for mobile and embedded devices.

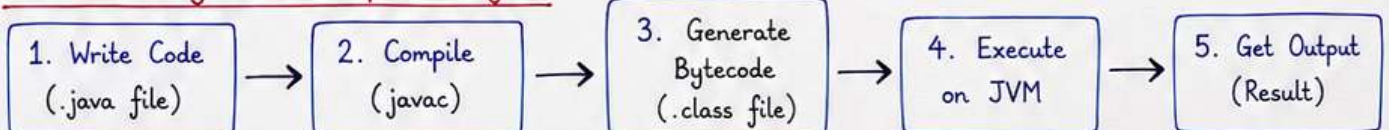
5. Example: Hello World in Java

```
// Hello World Program
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Explanation:

- class is a blueprint.
- main() is the entry point of program.
- println() prints the output on console.

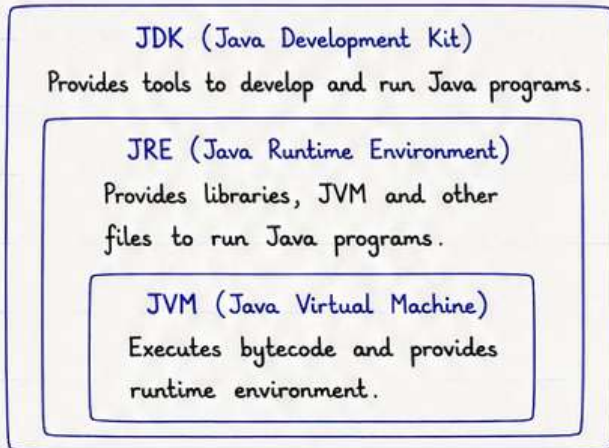
6. Java Program Development Cycle



COMPLETE JAVA NOTES

BEGINNER TO ADVANCED

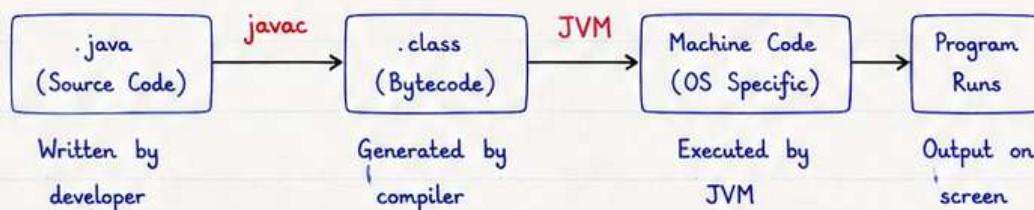
1. JDK, JRE and JVM



Key Points:

- JDK = JRE + Development Tools (javac, jar, javadoc, etc.)
- JRE = JVM + Libraries + Other files
- JVM = Loads .class files, verifies, executes and manages memory.
- We need JDK to develop and JRE to run Java applications.

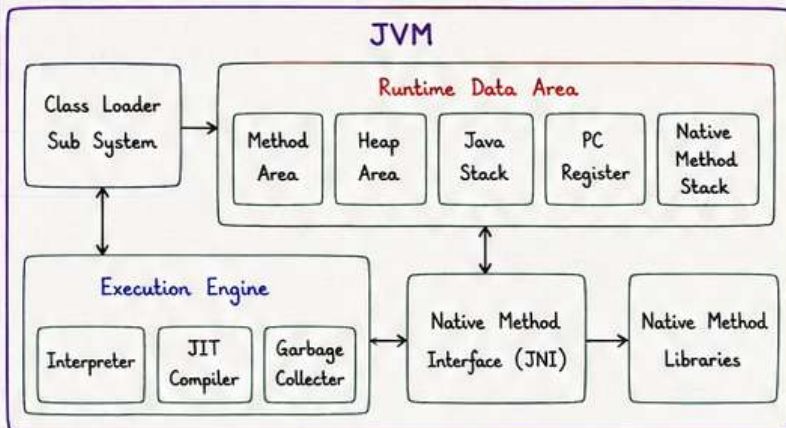
2. Compilation Process



Note:

Bytecode (.class) is platform independent. JVM makes it platform dependent.

3. JVM Architecture



4. Important Components

- Class Loader: Loads .class files into memory.
- Runtime Data Area: Stores data during program execution.
- Execution Engine: Executes bytecode.
- Interpreter: Executes code line by line.
- JIT Compiler: Compiles bytecode to machine code for faster execution.
- Garbage Collector: Frees unused memory automatically.
- JNI: Interface to interact with native code.

5. Java Memory Areas (Runtime Data Area)

Area	Description
Method Area	Stores class information, methods, static variables.
Heap Area	Stores objects and arrays. Shared by all threads.
Java Stack	Stores method calls and local variables. One stack per thread.
PC Register	Stores address of next instruction to be executed.
Native Method Stack	Used for native (C/C++) method calls.

6. Important Points

- Java is both compiled and interpreted.
- JVM provides platform independence.
- Garbage Collection makes Java memory safe.
- .java file is source code.
- .class file is bytecode.
- JVM executes bytecode.

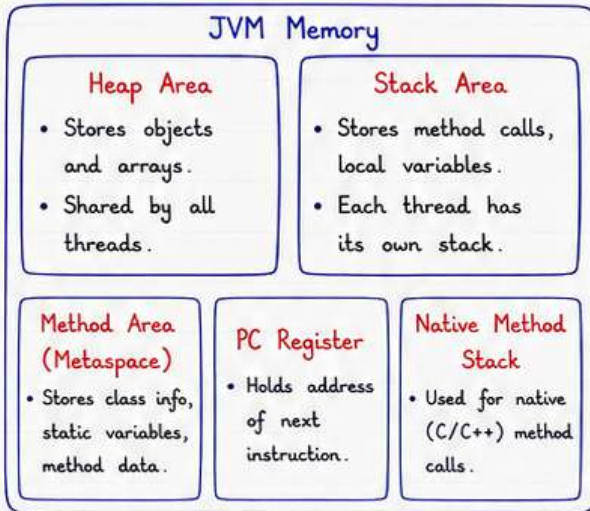
Pro Tip: Understand how Java works internally. It helps in debugging, performance tuning and crack any core Java interview question.

COMPLETE JAVA NOTES

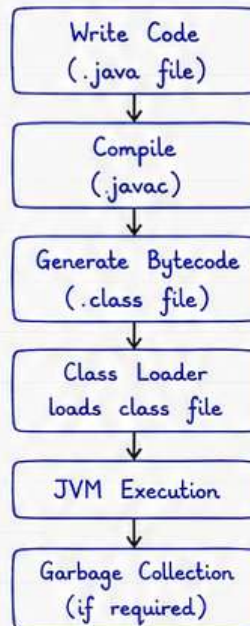
BEGINNER TO ADVANCED

1. Java Memory Model (JVM)

JVM manages memory for running Java programs.



2. Program Execution Flow



How it Works?

- Code is compiled to bytecode.
- JVM loads bytecode using Class Loader.
- Execution Engine runs the code.
- Memory is managed automatically.
- Garbage Collector removes unused objects.

3. Stack vs Heap

Feature	Stack	Heap
Nature	Stores primitive values and references.	Stores objects and arrays.
Access	Faster access.	Slower than stack.
Size	Smaller (limited).	Larger (dynamic).
Lifetime	Created when method is called, destroyed when it ends.	Exists till object is used, removed by GC.
Managed by	System (automatic).	Garbage Collector.
Example	int a = 10; String s = "Java"; (reference)	new Student(); int[] arr = new int[5];

4. Garbage Collection

- Garbage Collector (GC) removes objects that are no longer used.
- It frees heap memory automatically.
- Types of GC:
 - Serial GC
 - Parallel GC
 - G1 GC
 - ZGC (in later versions)

Why GC is needed?

- Prevents memory leaks.
- Improves application performance.
- Manages memory automatically.

5. Program Lifecycle in Java



- Create** : Write Java source code.
- Compile** : Convert .java file to .class file.
- Load** : Load class file into memory (by Class Loader).
- Link** : Verify bytecode, allocate memory.

- Initialize** : Initialize static variables and static blocks.
- Use** : Create objects and call methods.
- Unload** : Class is removed from memory when not used.

6. Common Errors

Error	Reason
StackOverflowError	Too many method calls (stack memory full).
OutOfMemoryError	Heap memory is full, objects cannot be created.
ClassNotFoundException	Class not found at runtime.

7. Quick Code Example

```

public class Demo {
    public static void main(String[] args) {
        int a = 10;           // Stack (primitive)
        String s = new String("Java"); // Reference in stack
                                // Actual object in heap
        System.out.println(a + " " + s);
    }
}
  
```

Output:

10 Java

Note:

Reference (s) is in stack, actual object lives in heap.

Pro Tip: Understand how JVM works, memory management and garbage collection. These are frequently asked in Java interviews and help in writing better and optimized code.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

★ OVERVIEW

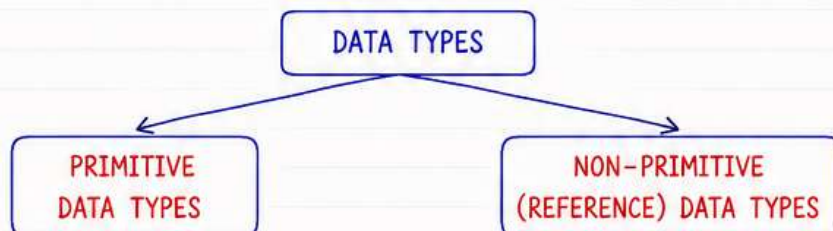
Java is a strongly typed language. It uses data types to define the type of data a variable can hold.

Java data types are divided into two categories.

- Primitive Data Types
- Non-Primitive (Reference) Data Types

★ DATA TYPES IN JAVA

There are two types of data types in Java.



1. PRIMITIVE DATA TYPES

Primitive data types are the basic building blocks in Java.

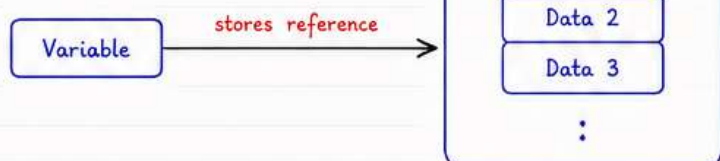
These types store actual values.

Type	Size	Default Value	Range / Description
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	-2^{31} to $2^{31} - 1$ (-2,147,483,648 to 2,147,483,647)
long	8 bytes	0L	-2^{63} to $2^{63} - 1$ (Very large range)
float	4 bytes	0.0f	Decimal numbers (6-7 digits precision)
double	8 bytes	0.0d	Decimal numbers (15-16 digits precision)
char	2 bytes	'\u0000'	0 to 65,535 (Unicode characters)
boolean	1 bit	false	true or false

2. NON-PRIMITIVE (REFERENCE) DATA TYPES

Non-primitive data types are used to store collections of data and more complex entities.

- String
- Arrays
- Classes
- Interfaces
- Collection Framework



Note : Java is a strongly typed language, meaning every variable must be declared with a specific type, and that type cannot be changed.

≡ JAVA FUNDAMENTALS NOTES ≡

2. OPERATORS IN JAVA

Key Point

Operators work on operands (variables or values) and produce a result.

Operators are special symbols that are used to perform **operations** on variables and values.

1. ARITHMETIC OPERATORS

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (Remainder)	10 % 3	1

Note : Division (/) between two integers gives integer result.
For decimal result, at least one operand should be **float/double**.

2. RELATIONAL (COMPARISON) OPERATORS

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or Equal to	5 >= 5	true
<=	Less than or Equal to	5 <= 4	false

Note : Relational operators return **boolean** values (true or false).

3. LOGICAL OPERATORS

Operator	Name	Meaning	Example	Result
&&	Logical AND	true if both conditions are true	(5 > 3) && (2 < 4)	true
	Logical OR	true if at least one condition is true	(5 > 3) (2 > 4)	true
!	Logical NOT	Reverses the result (true → false)	!(5 > 3)	false

Note : In Java, && and || use **short-circuit evaluation**.
• For **A && B** → If A is **false**, B is not evaluated.
• For **A || B** → If A is **true**, B is not evaluated.

4. ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5

Note : '=' is used to assign value.
Others are shorthand assignment operators.

5. UNARY OPERATORS

a) Increment (++)

Used to increase the value by 1.

- Pre-increment : ++x
- Post-increment : x++

Example	Operation	Result
int x = 5; ++x;	Pre-increment	x = 6
int x = 5; x++;	Post-increment	x = 6 (but uses 5 first)

b) Decrement (--)

Used to decrease the value by 1.

- Pre-decrement : --x
- Post-decrement : x--

Example	Operation	Result
int x = 5; --x;	Pre-decrement	x = 4
int x = 5; x--;	Post-decrement	x = 4 (but uses 5 first)

6. TERNARY (CONDITIONAL) OPERATOR

A shorthand way of writing **if-else** statement.

Syntax : condition ? expression1 : expression2

If condition is **true**,
expression1 is executed.

If condition is **false**,
expression2 is executed.

Example :

```
int age = 20;  
String result = (age >= 18) ? "Adult" : "Minor";
```

Output : Adult

7. OPERATOR PRECEDENCE (Higher precedence operators are evaluated first)

Precedence	Operators	Category	Example	Evaluates As
1	()	Parentheses	(3 + 2) * 5	(3 + 2) * 5
2	++, --, !, ~	Unary	++a + b	(++a) + b
3	*, /, %	Arithmetic	a + b * c	a + (b * c)
4	+, -	Arithmetic	a - b + c	(a - b) + c
5	<, <=, >, >=, instanceof	Relational	a > b && c < d	(a > b) && (c < d)
6	==, !=	Equality	a == b c != d	(a == b) (c != d)
7	&&	Logical AND	a && b c	(a && b) c
8		Logical OR		
9	?:	Ternary	a > b ? x : y	(a > b) ? x : y
10	=, +=, -=, *=, /=, %=	Assignment	a += 5	a += 5

QUICK RECAP

- ✓ Arithmetic → +, -, *, /, %
- ✓ Relational → ==, !=, >, <, >=, <=
- ✓ Logical → &&, ||, !
- ✓ Assignment → =, +=, -=, *=, /=, %=
- ✓ Unary → ++, --, !
- ✓ Ternary → ? :

JAVA FUNDAMENTALS NOTES

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

★ 1.1 PRIMITIVE DATA TYPES – MORE DETAILS

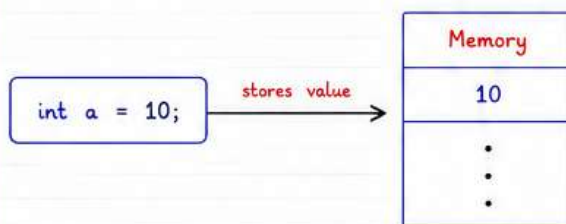
Primitive data types are predefined by Java and are not objects.

They store single values and are more efficient.

Type	Size	Default Value	Details
byte	1 byte	0	Stores whole numbers from -128 to 127.
short	2 bytes	0	Stores whole numbers from -32,768 to 32,767.
int	4 bytes	0	Most commonly used for whole numbers. Range: -2^{31} to $2^{31} - 1$
long	8 bytes	0L	Used for very large whole numbers. Range: -2^{63} to $2^{63} - 1$
float	4 bytes	0.0f	Stores decimal numbers with single precision. Sufficient for 6-7 decimal digits.
double	8 bytes	0.0d	Stores decimal numbers with double precision. Sufficient for 15-16 decimal digits.
char	2 bytes	'\u0000'	Stores a single Unicode character. Example: 'A', '1', '@'
boolean	1 bit	false	Stores logical values: true or false.

★ MEMORY ALLOCATION (CONCEPT)

Primitive types store the actual value in memory.



★ SIZE COMPARISON (BYTES)

byte	short	int	long	float	double
1	2	4	8	4	8

Size increasing

★ TYPE CATEGORIES SUMMARY

Primitive Data Types

- Store single values.
- Actual value is stored in memory.
- Have fixed size.
- Examples: int, float, char, boolean, etc.

Non-Primitive (Reference) Data Types

- Store collections of values.
- Store reference (address) in memory.
- No fixed size.
- Examples: String, Arrays, Classes, etc.

Note : Choose the appropriate data type to save memory and improve performance.

For example, use **int** instead of **long** if the number is within **int** range.

JAVA FUNDAMENTALS NOTES

1. DATA TYPES IN JAVA (CONTINUED)

NON-PRIMITIVE (REFERENCE) DATA TYPES

Non-primitive data types are more complex and are used to store collections of values or user-defined data. They store **references** (addresses) in **stack** memory, while actual data is stored in **heap** memory.

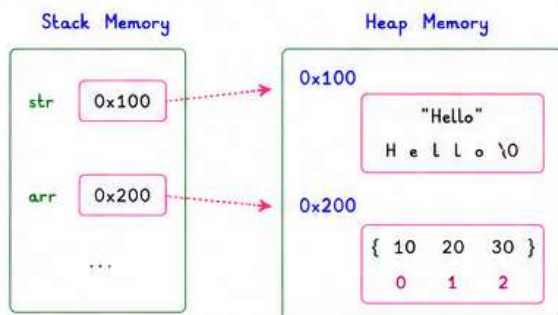
TYPES OF NON-PRIMITIVE DATA TYPES

Type	Description	Example	Default Value	Stored In	Example Code
String	Sequence of characters. Used to store text.	"Hello", "Java", "123"	null	Heap	String name = "Java"; String msg = new String("Hello"); System.out.println(name);
Array	Stores fixed-size sequence of elements of same type.	int[] arr = {1, 2, 3}; char[] ch = {'a', 'b'};	null	Heap	int[] arr = {10, 20, 30}; System.out.println(arr[1]); // 20
Class	User-defined blueprint to create objects.	Student s = new Student();	null	Heap	Student s = new Student("Ram", 21); s.display();
Interface	Reference type that contains abstract methods.	List<String> list;	null	Heap	List<String> list = new ArrayList<>(); list.add("A");
Enum	Special data type that consists of a fixed set of constants.	Day d = Day.MONDAY;	null	Heap	enum Day { MONDAY, TUESDAY, WEDNESDAY } Day d = Day.TUESDAY;

Key Points

- Default value of all reference types is null.
- Created using classes, constructors or literals.
- Support more functionality and are memory efficient for large data.

STACK VS HEAP MEMORY (REFERENCE TYPES)



Variables in stack hold reference (address) to actual data present in heap.

NULL IN REFERENCE TYPES

A reference variable can point to an object or be null.

Example :

```
String s = null; // s does not point to any object
int[] arr = null; // arr does not point to any array
Student s1 = null; // s1 does not point to any object
```



If you try to access a null reference, it causes **NullPointerException** at runtime.

EXAMPLE PROGRAM (USING NON-PRIMITIVE TYPES)

```
class Student {
    String name; // non-primitive (String)
    int age; // primitive

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println(name + " is " + age + " years old");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("John", 20); // class (object)
        s1.display();

        String city = "Delhi"; // String
        int[] marks = {85, 90, 78}; // Array
        System.out.println(city);
        System.out.println(marks[1]);
    }
}
```

Output :

```
John is 20 years old
Delhi
90
```

QUICK RECAP

- ✓ Non-primitive data types store references, not actual values.
- ✓ They are stored in **heap** memory.
- ✓ Default value of all non-primitive types is null.
- ✓ Examples : String, Array, Class, Interface, Enum.
- ✓ They provide more features and help in writing structured and reusable code.

WHEN TO USE?

- Use String to store text.
- Use Array to store multiple values of same type.
- Use Class to create objects for real-world entities.
- Use Interface for abstraction.
- Use Enum for fixed set of constants.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ LOOPS IN JAVA ≡

Loops are used to execute a block of code multiple times based on a condition.

Java provides 3 types of loops.

- for loop
- while loop
- do-while loop

Why Loops?

- To avoid repetition
- To iterate over a range
- To traverse arrays, strings, collections, etc.

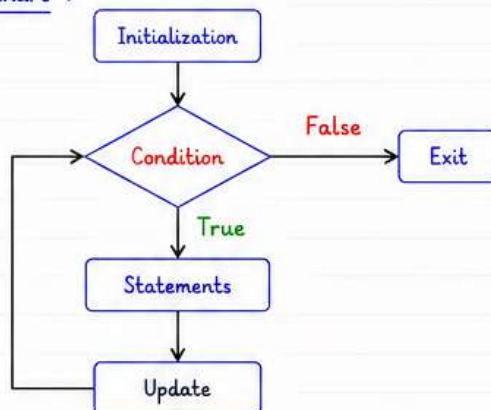
★ 1. FOR LOOP

The for loop is used when the number of iterations is known in advance.

Syntax :

```
for (initialization; condition; update) {  
    // statements  
}
```

Flowchart :



Example : Print numbers from 1 to 5

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

How it works?

- ① Initialize $i = 1$
- ② Check condition $i \leq 5 \rightarrow \text{True}$
- ③ Execute statements (print i)
- ④ Update $i \rightarrow i = i + 1$
- ⑤ Repeat steps ② to ④ until condition becomes **False**

Output :

```
1  
2  
3  
4  
5
```

★ COMPARISON OF LOOPS

Feature	for loop	while loop	do-while loop
When to use	When number of iterations is known	When number of iterations is unknown	When block must execute at least once
Condition Check	Before executing the loop	Before executing the loop	After executing the loop
Execution	0 or more times	0 or more times	1 or more times
Syntax	More compact	Less compact	Medium
Use Case	Counting, traversing, arrays, fixed iterations	User input validation, unknown iterations	Menu driven programs, atleast one execution

- Note :**
- Avoid infinite loops (condition should eventually become false).
 - Use loops carefully to improve performance and readability of code.

JAVA FUNDAMENTALS NOTES

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

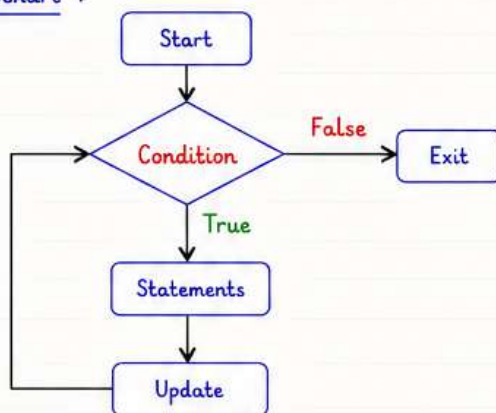
★ 2. WHILE LOOP

The **while** loop is used when the number of iterations is not known in advance.

Syntax :

```
while (condition) {  
    // statements  
    // update  
}
```

Flowchart :



Example : Print numbers from 1 to 5

```
int i = 1;  
while (i <= 5) {  
    System.out.println(i);  
    i = i + 1;  
}
```

How it works?

- ① Initialize $i = 1$
- ② Check condition $i \leq 5 \rightarrow \text{True}$
- ③ Execute statements (print i)
- ④ Update $i \rightarrow i = i + 1$
- ⑤ Repeat steps ② to ④ until condition becomes **False**

Output :

```
1  
2  
3  
4  
5
```

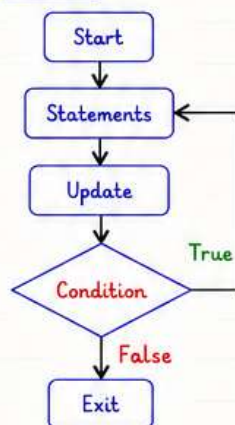
★ DO-WHILE LOOP

The **do-while** loop is similar to while loop, but the block of code is executed **at least once**.

Syntax :

```
do {  
    // statements  
    // update  
} while (condition);
```

Flowchart :



Example : Print numbers from 1 to 5

```
int i = 1;  
do {  
    System.out.println(i);  
    i = i + 1;  
} while (i <= 5);
```

Output :

```
1  
2  
3  
4  
5
```

★ COMPARISON OF WHILE vs DO-WHILE

Feature	while loop	do-while loop
Execution	May execute zero times	Executes at least once
Condition Check	Before executing the block	After executing the block
Syntax	while (condition) { ... }	do { ... } while (condition);
Use Case	When iterations may be zero	When at least one execution is required

Note : Use **while** when the condition is checked first.

Use **do-while** when the block must run at least once.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ LOOPS IN JAVA (CONTINUED) ≡

★ 3. FOR-EACH LOOP (ENHANCED FOR LOOP)

The for-each loop is used to iterate over arrays or collections easily.

Syntax :

```
for (datatype variable : array/collection) {  
    // statements  
    // variable takes each element  
}
```

Example 1 : Using array

```
int[] arr = {10, 20, 30, 40, 50};  
for (int num : arr) {  
    System.out.println(num);  
}
```

Output :

10 20 30 40 50

Example 2 : Using ArrayList

```
ArrayList<String> names = new ArrayList<>();  
names.add("Aman");  
names.add("Riya");  
names.add("Vikram");  
  
for (String name : names) {  
    System.out.println(name);  
}
```

Output :

Aman
Riya
Vikram

Important Points

- Works with arrays and collections.
- No need to use index.
- Read-only access (cannot modify elements directly).
- Cleaner and easier syntax.

★ COMPARISON: FOR LOOP vs FOR-EACH LOOP

Feature	For Loop	For-Each Loop (Enhanced For Loop)
Syntax	More verbose	Simpler and cleaner
Access	Provides index (can access position)	Does not provide index
Use Case	When index is needed or we want to modify elements	When we only need to read elements
Works With	Arrays, Collections, Strings	Arrays, Collections only
Modification	Can modify elements using index	Cannot modify elements directly
Flexibility	More flexible	Less flexible (read-only)

★ OTHER IMPORTANT LOOP NOTES

Break Statement

Used to exit the loop immediately.

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
    System.out.println(i);  
}
```

Output :

1 2

Continue Statement

Skips the current iteration and moves to next.

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    System.out.println(i);  
}
```

Output :

1 2 4 5

Nested Loops

A loop inside another loop.

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 2; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

Output :

* *
* *
* *

Note : Choose the right loop based on the requirement for better performance and readability of your code.



≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ BREAK and CONTINUE in LOOPS ≡

Both **break** and **continue** are used to control the flow of loops.

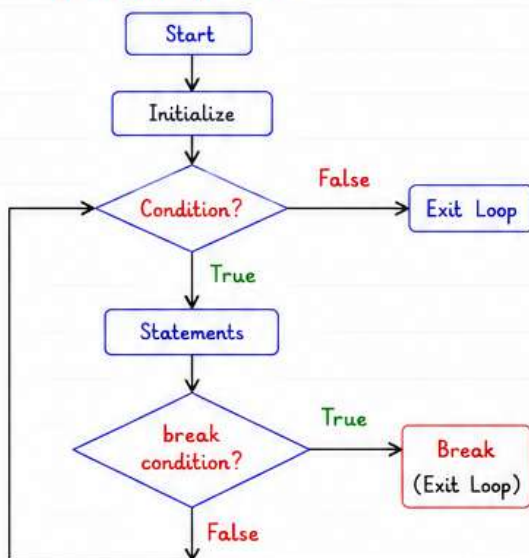
BREAK KEYWORD

- Terminates the loop immediately.
- Control comes out of the loop.
- Remaining iterations are skipped.

When to use?

Use **break** when a certain condition is met and we want to **exit the loop**.

Flowchart of break



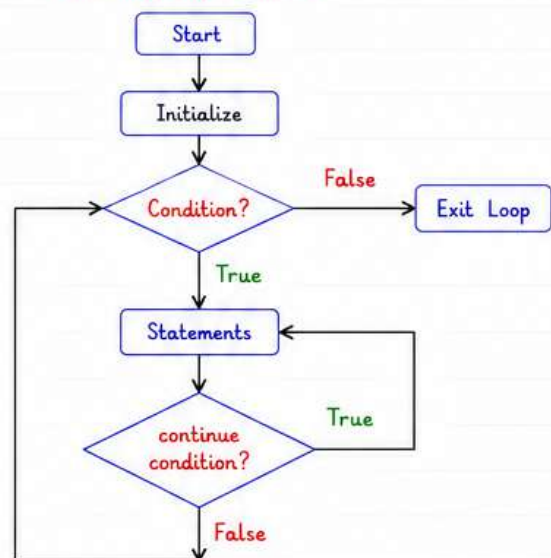
CONTINUE KEYWORD

- Skips the current iteration.
- Control moves to the next iteration.
- Remaining iterations are executed.

When to use?

Use **continue** when a certain condition is met and we want to **skip the current iteration**.

Flowchart of continue



EXAMPLE 1: Using break

```
// Print numbers from 1 to 10, but stop when number is 6
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break; // exit the loop
    }
    System.out.print(i + " ");
}
```

Output: 1 2 3 4 5

EXAMPLE 2: Using continue

```
// Print odd numbers from 1 to 10 (skip even numbers)
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // skip even numbers
    }
    System.out.print(i + " ");
}
```

Output: 1 3 5 7 9

WHEN TO USE?

Use **break** When

- We want to terminate the loop completely.
- A condition is met and further iterations are not required.
- Example: searching an element and found it.



Use **continue** When

- We want to skip some iterations.
- A condition is met, but remaining iterations are still needed.
- Example: skipping even numbers, invalid inputs, etc.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ CONDITIONAL STATEMENTS IN JAVA ≡

Conditional statements are used to make decisions in a program based on certain conditions.

Java provides the following conditional statements:

- if statement
- if-else statement
- else-if ladder
- nested if statement
- switch statement

Key Points

- Conditions must evaluate to **boolean** (true or false).
- If a condition is **true**, the corresponding block is executed.
- If a condition is **false**, the next possible condition is checked.
- If no condition is **true**, **else** block (if present) is executed.

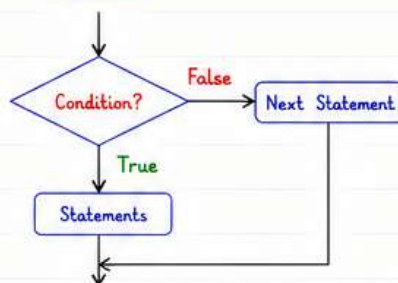
★ 1. if STATEMENT

Executes a block of code if the condition is true.

Syntax :

```
if (condition) {  
    // statements  
}
```

Flowchart :



Example :

```
int age = 18;  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
}
```

Output :

Eligible to vote

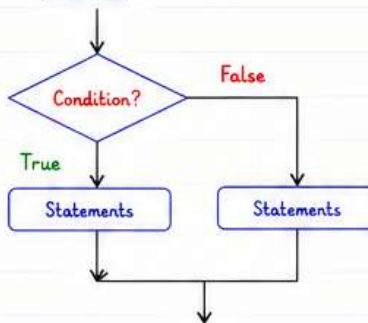
★ 2. if-else STATEMENT

Executes one block if condition is true otherwise executes another block.

Syntax :

```
if (condition) {  
    // statements  
} else {  
    // statements  
}
```

Flowchart :



Example :

```
int age = 16;  
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Output :

Minor

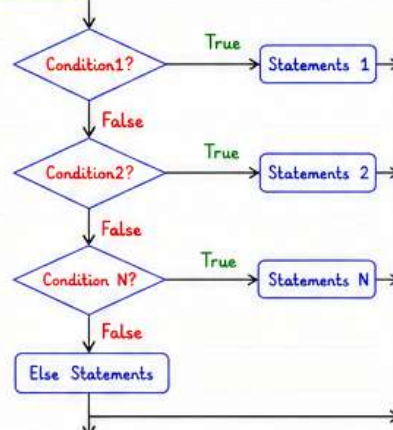
★ 3. else-if LADDER

Used to check multiple conditions.

Syntax :

```
if (condition1) {  
    // statements  
} else if (condition2) {  
    // statements  
} else if (condition3) {  
    // statements  
} else {  
    // statements  
}
```

Flowchart :



Example :

```
int marks = 75;  
if (marks >= 90) {  
    System.out.println("Grade A");  
} else if (marks >= 75) {  
    System.out.println("Grade B");  
} else if (marks >= 50) {  
    System.out.println("Grade C");  
} else {  
    System.out.println("Fail");  
}
```

Output :

Grade B

Note : Only the first **true** condition block is executed.
Once a block is executed, the remaining conditions are skipped.



≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ CONDITIONAL STATEMENTS IN JAVA (CONTINUED) ≡

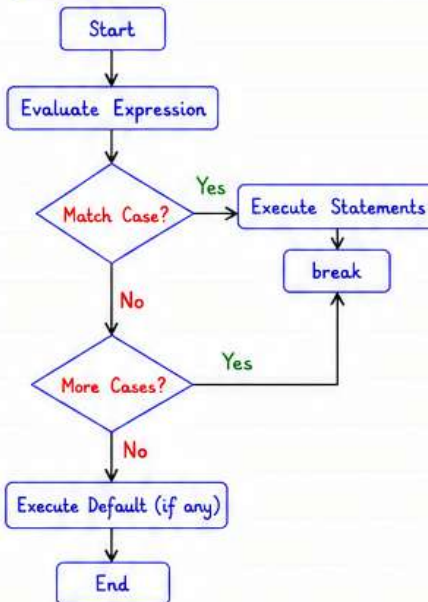
★ 4. SWITCH STATEMENT

The `switch` statement is used to execute one block of code from multiple alternatives.

Syntax :

```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    ...  
    default:  
        // statements  
}
```

Flowchart :



Example :

```
int day = 3;  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Invalid day");  
}
```

Output :

Wednesday

IMPORTANT POINTS ABOUT SWITCH

- The expression must be of type: byte, short, int, char, String, enum.
- Each case value must be unique.
- `break` is important to exit the switch. Without `break`, execution falls through to the next case.
- `default` is optional and executed when no case matches.
- Since Java 14, switch can be used as an expression (new feature).

FALL-THROUGH EXAMPLE (WITHOUT BREAK)

```
int num = 2;  
switch (num) {  
    case 1: System.out.println("One");  
    case 2: System.out.println("Two");  
    case 3: System.out.println("Three");  
    default: System.out.println("Other");  
}
```

Output :

Two
Three
Other

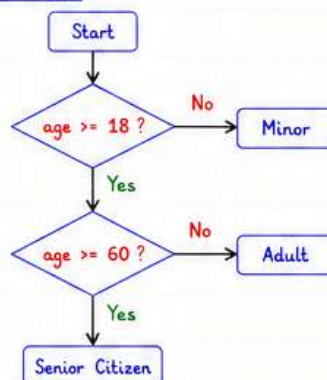
★ 5. NESTED IF STATEMENT

An if statement inside another if or else block.

Example :

```
int age = 20;  
if (age >= 18) {  
    if (age >= 60) {  
        System.out.println("Senior Citizen");  
    } else {  
        System.out.println("Adult");  
    }  
} else {  
    System.out.println("Minor");  
}
```

Flowchart :



QUICK RECAP

- ☑ if → for one condition
- ☑ if-else → for two conditions
- ☑ else-if ladder → for multiple conditions
- ☑ switch → for multiple fixed values
- ☑ Nested if → for complex conditions



Note : Use if-else when conditions are range based or complex.

Use switch when you have many fixed choices of the same variable.

COMMENT PDF

— TO GET —

COMPLETE PDF

OF

COLLECTION

JAVA COLLECTIONS FRAMEWORK

TYPES OF COLLECTIONS

-  LIST
-  SET
-  MAP
-  QUEUE
-  STACK
-  COMPARATOR & COMPARABLE

WHY COLLECTIONS ?

-  EFFICIENT DATA HANDLING
-  REUSABILITY
-  BETTER PERFORMANCE
-  DATA STRUCTURE SUPPORT
-  INTEROPERABILITY
-  SCALABILITY



SAVE



SHARE



FOLLOW



aman_views



OOPS CONCEPTS

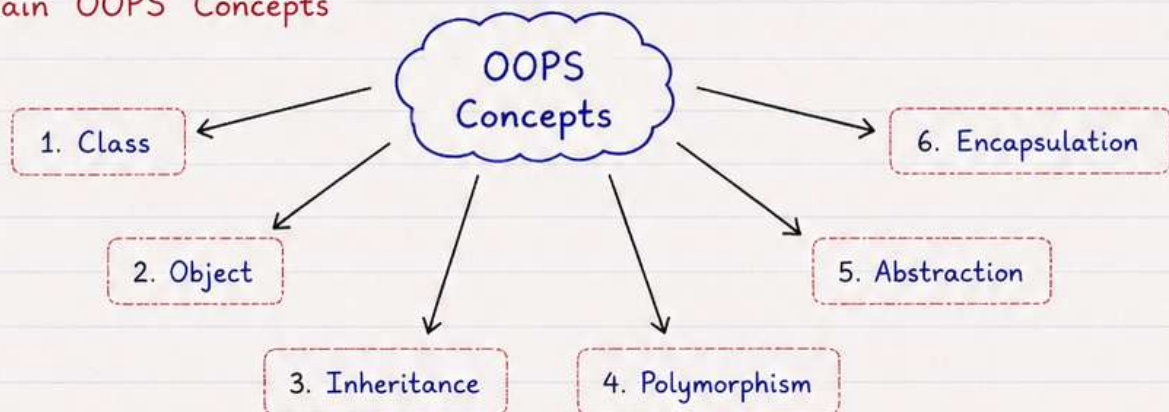
★ What is OOPS ?

OOPS (Object-Oriented Programming System) is a programming paradigm based on the concept of "Objects", which can contain data in the form of fields (attributes) and code in the form of procedures (methods).

★ Why OOPS ?

- It makes programs easier to understand, maintain and modify.
- It provides data security by data hiding.
- It supports reusability through inheritance.
- It allows flexibility through polymorphism.
- It helps in modeling real-world entities.

★ Main OOPS Concepts



★ OOPS in Real World



★ Summary

OOPS is a powerful programming paradigm that uses objects and classes to design modular, reusable and scalable applications. The above concepts work together to make the code efficient and easy to manage.

Remember:

Think Classes,
Create Objects,
Build Solutions! ☆

★ ⌢ OOPS CONCEPT FULL HANDBOOK ⌢ ★

⌢ 1. CLASS ⌢

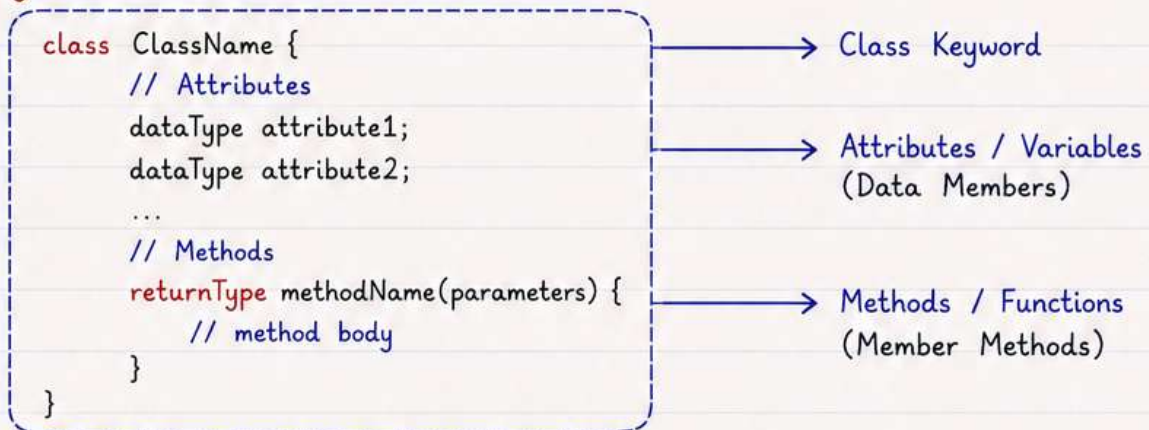
★ What is a Class ?

A class is a **blueprint** or **template** for creating objects. It defines a set of **attributes** (data) and **methods** (functions) that the objects of the class will have.

★ Key Points

- A class does **not** occupy memory.
- It is a **user-defined** data type.
- It acts as a **blueprint** for creating objects.
- It helps in **data hiding** and **code reusability**.

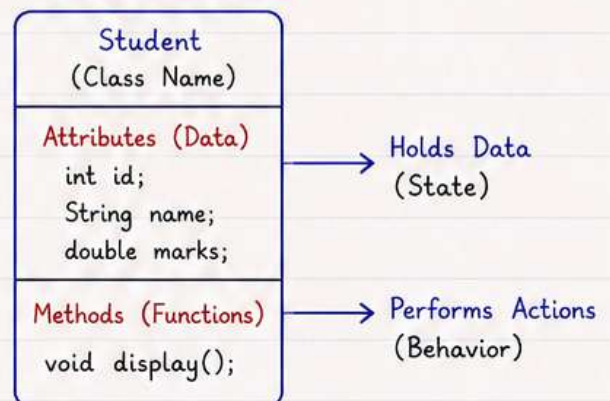
★ Syntax (in Java)



★ Example

```
class Student {  
    // Attributes  
    int id;  
    String name;  
    double marks;  
    // Methods  
    void display() {  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
        System.out.println("Marks: " + marks);  
    }  
}
```

Class Structure



Remember :

A class is just a **template**. No memory is allocated when a class is created. Memory is allocated **only** when objects are created from the class.

Example :

```
Student s1 = new Student();  
// Object creation
```

☆ ≧ OOPS CONCEPT FULL HANDBOOK ≦ ☆

≧ 2. OBJECT ≦

★ What is an Object ?

An object is a real-world entity which is an instance of a class.
It has state (data) and behavior (methods).

★ Key Points

- An object is created from a class.
- It occupies memory.
- Each object has its own copy of data.
- Objects can interact by calling methods.
- Multiple objects can be created from the same class.

★ Syntax (in Java)

```
ClassName objectName = new ClassName();
```

→ Class Name

→ Object Name (Reference)

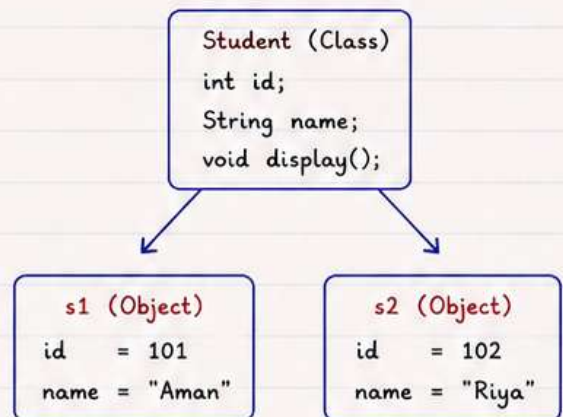
→ new keyword

→ Constructor

★ Example

```
class Student {  
    int id;  
    String name;  
  
    void display() {  
        System.out.println(id + " - " + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // Creating objects  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        // Setting values  
        s1.id = 101;  
        s1.name = "Aman";  
        s2.id = 102;  
        s2.name = "Riya";  
  
        // Calling method  
        s1.display(); // Output: 101 - Aman  
        s2.display(); // Output: 102 - Riya  
    }  
}
```

Object Diagram



Here, s1 and s2 are objects (instances) of class Student.
Each object has its own data.



Remember :

- Class is a blueprint.
- Object is a real entity created from the class.
- Many objects can be created from one class.

Example in Real Life :

Class : Mobile

Objects : iphone, samsung, oneplus (each mobile has its own details and behavior)

★ OOPS CONCEPT FULL HANDBOOK ★

3. METHOD

★ What is a Method ?

A method is a block of code that performs a specific task. It is defined inside a class and is used to define the behavior of objects.

★ Key Points

- A method is a function that belongs to a class.
- It is used to perform some operation.
- It can access and modify the data (attributes) of the class.
- Methods improve code modularity and reusability.
- Methods can return a value or may not return anything (void).

★ Syntax (in Java)

```
returnType methodName(parameters) {  
    // method body  
    // statements  
    return value; // optional  
}
```

Return type of method
(int, void, String, etc.)

Name of the method

Parameters passed to the method
(optional)

Method body

★ Example

```
class Calculator {  
    int add(int a, int b) {  
        int sum = a + b;  
        return sum;  
    }  
    void showMessage() {  
        System.out.println("Hello, Java!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        int result = c.add(10, 20); // calling method with return  
        System.out.println("Sum = " + result);  
        c.showMessage();           // calling void method  
    }  
}
```

How Method Works ?



Object calls the method.

Method performs the task.

Method returns the result (if any).

Remember :

- Method is the behavior of an object.
- It is always defined inside a class.
- A method can return a value or nothing.
- Good method names describe what the method does.

Example in Real Life :

Think of a remote control.

Pressing a button (method) performs an action like changing channel, increasing volume, etc.

Each button has a specific function.



4. CONSTRUCTOR

★ What is a Constructor ?

A constructor is a special method in a class that is used to **initialize objects**. It is called **automatically** when an object is created.

★ Key Points

- Constructor name is **same as class name**.
- It has **no return type**, not even void.
- It is called **automatically** when object is created.
- Used to **initialize** object's data.
- A class can have **multiple** constructors (constructor overloading).
- If no constructor is written, Java provides a **default constructor**.

★ Syntax (in Java)

```
class ClassName {  
    ClassName(parameters) {  
        // initialization code  
    }  
}
```

→ Constructor name

→ Parameters (optional)

→ Constructor body

★ Example

```
class Student {  
    int id;  
    String name;  
    // Constructor  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
    void display() {  
        System.out.println(id + " - " + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student(101, "Aman");  
        Student s2 = new Student(102, "Riya");  
        s1.display();    // Output: 101 - Aman  
        s2.display();    // Output: 102 - Riya  
    }  
}
```

How Constructor Works ?

new Student(101, "Aman")

↓
Constructor is called

↓
Object is created

↓
Data is initialized

Types of Constructors

- Default Constructor (no-arg)
- Parameterized Constructor
- Copy Constructor (via parameter)

Remember :

- Constructor runs only **once** when object is created.
- It helps to **initialize** object in a clean way.
- Constructors have **no return type**.

Example in Real Life :

When you register in any website, your **details** (name, email, password) are filled automatically in the form. Constructor works the same way.

☆ ≡ OOPS CONCEPT FULL HANDBOOK ≡ ☆

≡ 5. INHERITANCE ≡

★ What is Inheritance ?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent/superclass). It promotes code reusability and establishes an IS-A relationship.

★ Key Points

- The class whose members are inherited is called **superclass** (parent class).
- The class that inherits is called **subclass** (child class).
- The subclass can have its **own members** in addition to inherited members.
- Inheritance supports **code reusability** and **extensibility**.
- Java **does not support multiple inheritance** with classes (but supports it using interfaces).

★ Syntax (in Java)

```
class SuperClass {  
    // fields and methods  
}
```

→ Parent / Superclass

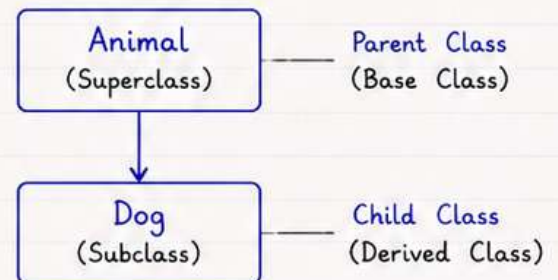
```
class SubClass extends SuperClass {  
    // own fields and methods  
}
```

→ Child / Subclass

★ Example (in Java)

```
// Parent Class (Superclass)  
class Animal {  
    String color = "Brown";  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
// Child Class (Subclass)  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
// Main Class  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();    // inherited method  
        d.bark();   // own method  
        System.out.println(d.color); // inherited field  
    }  
}
```

Inheritance Diagram



How it Works ?

- ①. Dog is a Animal. (IS-A Relationship)
- ②. Dog inherits color and eat() from Animal.
- ③. Dog can use inherited members and also define its own members.

Benefits

- Code Reusability
- Easy Maintenance
- Extensibility
- Better Organization

Remember

- ✓ Use **extends** keyword to inherit.
- ✓ Private members are not inherited.
- ✓ Constructors are not inherited.
- ✓ Inheritance represents **IS-A** relationship.



6. POLYMORPHISM

★ What is Polymorphism ?

Polymorphism means "many forms". It allows one interface to be used for different types.

In Java, polymorphism is achieved by Method Overloading and Method Overriding.

★ Types of Polymorphism

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

1. Compile-time Polymorphism (Method Overloading)

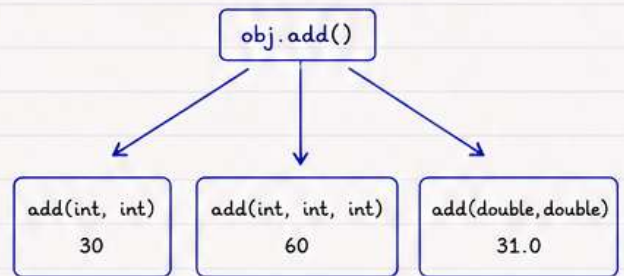
- Occurs in the same class.
- Same method name but different parameter list (number, type or order).
- It is resolved at compile time.

Example (in Java)

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator obj = new Calculator();
        System.out.println(obj.add(10, 20)); // 30
        System.out.println(obj.add(10, 20, 30)); // 60
        System.out.println(obj.add(10.5, 20.5)); // 31.0
    }
}
```

How it Works ?



The compiler decides which method to call based on the arguments passed.

2. Run-time Polymorphism (Method Overriding)

- Occurs in different classes using inheritance.
- Same method name, same parameters.
- It is resolved at runtime.

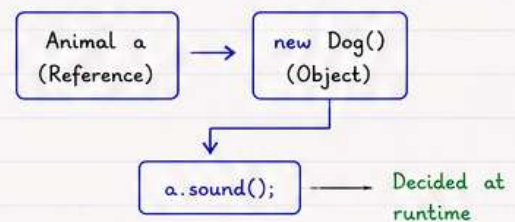
Example (in Java)

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Reference of parent, object of child
        a.sound(); // Calls overridden method
    }
}
```

How it Works ?



JVM decides at runtime which method to call based on the actual object.

Key Difference

Feature	Method Overloading	Method Overriding
Type	Compile-time	Run-time
Where	Same class	Different class (Inheritance)
Parameters	Different parameter list	Same parameters
Binding	Static binding	Dynamic binding

Remember

- Overloading increases readability.
- Overriding achieves runtime polymorphism.
- @Override annotation is good practice.
- Polymorphism improves code reusability and flexibility.



OOPS CONCEPT FULL HANDBOOK



8. ENCAPSULATION

★ What is Encapsulation ?

Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of the object's components. It helps in data hiding and protects the data from unauthorized access.

★ Key Points

- We achieve encapsulation by using access modifiers.
- Class variables should be private.
- Access to variables is provided through public getter and setter methods.
- It protects data from being modified by accident.
- It improves code maintainability and flexibility.

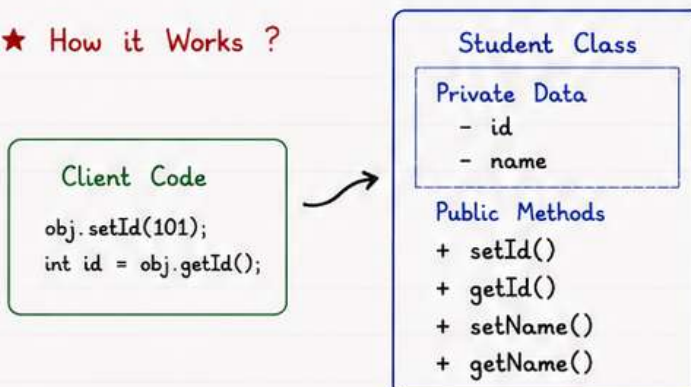
★ Syntax (in Java)

```
class ClassName {  
    private dataType variable; // private variable  
  
    // getter method  
    public dataType getVariable() {  
        return variable;  
    }  
  
    // setter method  
    public void setVariable(dataType value) {  
        variable = value;  
    }  
}
```

★ Example (in Java)

```
class Student {  
    private int id;  
    private String name;  
  
    // Getter for id  
    public int getId() {  
        return id;  
    }  
  
    // Setter for id  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    // Getter for name  
    public String getName() {  
        return name;  
    }  
  
    // Setter for name  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

★ How it Works ?



Benefits

- ✓ Protects data from unauthorized access.
- ✓ Improves security.
- ✓ Allows controlled access to data.
- ✓ Makes code easy to maintain and modify.

Remember

- ✓ Make variables private.
- ✓ Provide public getter and setter methods.
- ✓ Access and modify data only through methods.

☆ ≡ OOPS CONCEPT FULL HANDBOOK ≡ ☆

≡ 9. SUMMARISE ALL 4 OOPS PILLARS ≡

OOPs is built on four main pillars. Together, they help us design robust, reusable and maintainable software.

Pillar	Meaning	Key Points	Example (in Java)
1. Encapsulation Wrapping data and methods together as a single unit and restricting direct access.	It protects the data and hides internal details using access modifiers .	- Achieved using private variables. - Access provided through public getter and setter methods. - Improves security and maintainability.	<pre>class Student { private int id; private String name; public int getId() { return id; } }</pre>
2. Abstraction Hiding implementation details and showing only essential features.	It focuses on what an object does instead of how it does it.	- Achieved using abstract classes and interfaces. - Reduces complexity. - Helps in defining contracts.	<pre>abstract class Shape { abstract void draw(); } class Circle extends Shape { void draw() { // implementation } }</pre>
3. Inheritance Acquiring properties and behaviors of one class into another class.	It promotes code reusability and establishes an IS-A relationship.	- Achieved using extends (class) and implements (interface). - Child class inherits fields and methods of parent class. - Supports method overriding.	<pre>class Animal { void eat() { } } class Dog extends Animal { void bark() { } }</pre>
4. Polymorphism Ability of an object to take many forms.	It allows one interface to be used for different types.	- Achieved through method overloading and method overriding. - Increases flexibility. - Supports compile-time and run-time polymorphism.	<pre>class Animal { void sound() { } } class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }</pre>

In Short

- ✓ Encapsulation protects data and restricts access.
- ✓ Abstraction hides details and shows only essential features.
- ✓ Inheritance allows code reusability through IS-A relationship.
- ✓ Polymorphism allows one interface to work with multiple forms.

Together, these four pillars:

- Make code reusable
- Reduce complexity
- Improve flexibility
- Increase security and maintainability.

≡ COMMENT ☆

≡ PDF ≡

— TO GET —

≡ COMPLETE PDF ≡

≡ OF ≡

OOPS

OBJECT ORIENTED PROGRAMMING SYSTEM

4 PILLARS OF OOPS



ENCAPSULATION



ABSTRACTION



INHERITANCE



POLYMORPHISM

WHY OOPS ?



REUSABILITY



MODULARITY



FLEXIBILITY



EASY MAINTENANCE



SCALABILITY



SAVE



SHARE



FOLLOW



aman_views





1. COLLECTIONS OVERVIEW



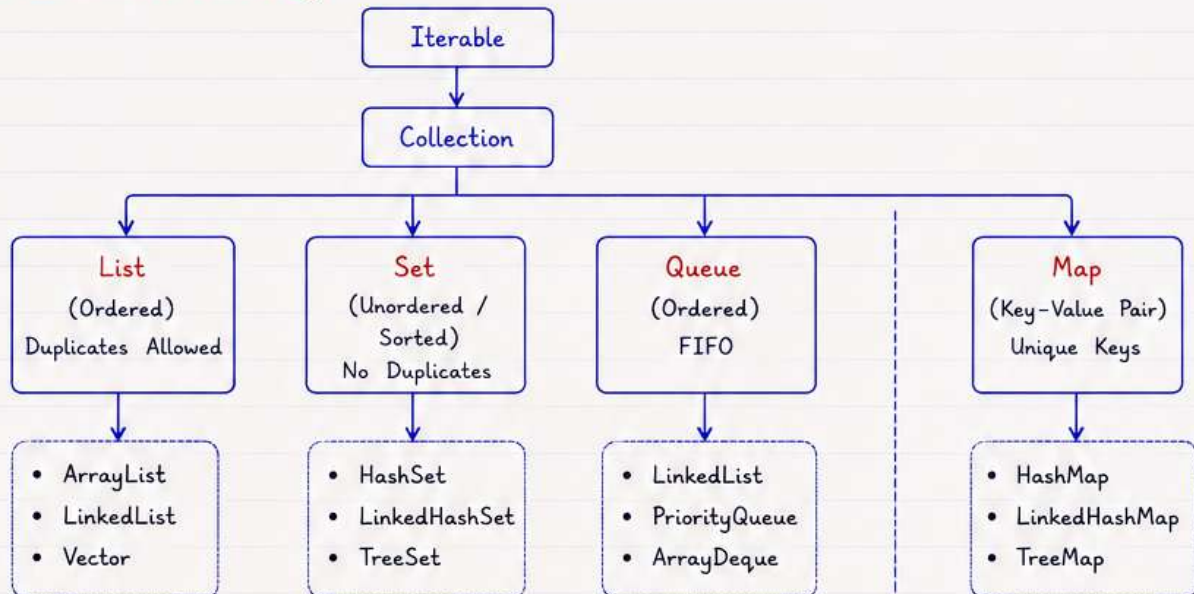
★ What is Collections Framework ?

Java Collections Framework is a **architecture** that provides ready-made classes and interfaces to store and manipulate groups of objects.

Why Collections ?

- To store multiple objects.
- To manipulate and process data.
- To reduce manual resizing.
- To provide better performance and easy to maintain code.

★ Collections Hierarchy



★ Quick Comparison

Feature	List	Set	Queue	Map
Order	Maintains Order	No Order / Sorted (TreeSet)	Maintains Order (FIFO)	No Order (Key-Value)
Duplicates	Allowed	Not Allowed	Allowed	Keys - Not Allowed Values - Allowed
Null Elements	Multiple Nulls Allowed	1 Null Allowed (HashSet)	1 Null Allowed (LinkedList)	1 Null Key (HashMap) Multiple Null Values
Access	Index Based	No Index	Sequential Access	Key Based
Performance	Fast Access	Fast Search (HashSet)	Depends on Implementation	Fast Lookup (HashMap)
Use Case	Store Ordered Collection	Store Unique Elements	Process in Sequence	Store Key-Value Data

★ Key Takeaway

- Use **List** when you need **ordered** collection with duplicates.
- Use **Set** when you need only **unique** elements.
- Use **Queue** when you need to process elements in sequence.
- Use **Map** when you need to store data in **key-value** pair.

Collections Framework



Powerful, Flexible
& Easy to Use



2. LIST INTERFACE



★ What is List ?

List is an **ordered** collection that allows **duplicate** elements. Elements are stored in a specific order and can be accessed by their **index**.

List Example

0	1	2	3	4
10	20	20	30	40

Duplicates Allowed, Order Maintained,
Index Based

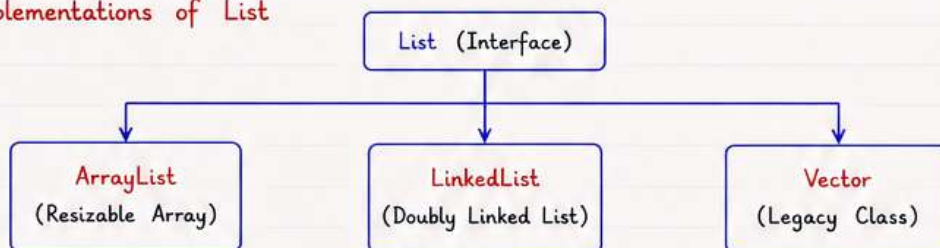
★ Features of List

- Maintains **insertion** order.
- Allows duplicate elements.
- Allows null values.
- Elements can be accessed by **index**.
- Useful when order of elements is important.

★ Common Methods

- `add(E e)` → Adds element
- `add(int index, E e)` → Adds at specific index
- `get(int index)` → Returns element
- `set(int index, E e)` → Updates element
- `remove(int index)` → Removes element
- `contains(Object o)` → Checks presence
- `size()` → Returns size

★ Implementations of List



★ List Implementations Comparison

Feature	ArrayList	LinkedList	Vector
Underlying Data Structure	Resizable Array	Doubly Linked List	Resizable Array
Order	Maintains Insertion Order	Maintains Insertion Order	Maintains Insertion Order
Duplicates	Allowed	Allowed	Allowed
Null Elements	Allowed	Allowed	Allowed
Random Access	Fast ($O(1)$)	Slow ($O(n)$)	Fast ($O(1)$)
Insertion/Deletion	Slow in middle ($O(n)$)	Fast ($O(1)$)	Slow in middle ($O(n)$)
Thread Safe	No	No	Yes
Best Use	More Read Operations	More Insert/Delete Operations	Legacy (Thread Safe) Requirements

★ Example (ArrayList)

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(20); // duplicates allowed
list.add(30);
System.out.println(list.get(2)); // 20
System.out.println(list);        // [10, 20, 20, 30]
```

Use List when :

- You need ordered collection.
- You need to allow duplicates.
- You need index based access.



3. ARRAYLIST



★ What is ArrayList ?

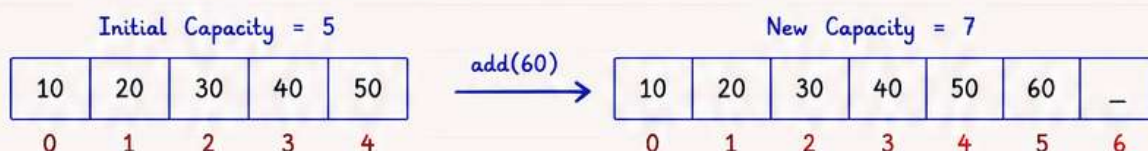
ArrayList is a **resizable-array** implementation of the **List** interface. It uses a dynamic array to store elements and grows automatically when the capacity is full.

Key Features

- Implements **List** interface.
- **Ordered** collection (maintains insertion order).
- Allows **duplicate** elements.
- Allows **null** elements.
- Not synchronized (not thread-safe).
- Fast **random access** (index based).

★ Internal Working

ArrayList internally uses a dynamic array. When the array becomes full, a new bigger array is created (usually 1.5 times) and all elements are copied to the new array.



★ Time Complexity

Operation	Complexity
get(index)	O(1)
add(E e) (at end)	O(1) amortized
add(index, E e)	O(n)
remove(index)	O(n)
contains(Object o)	O(n)
size()	O(1)

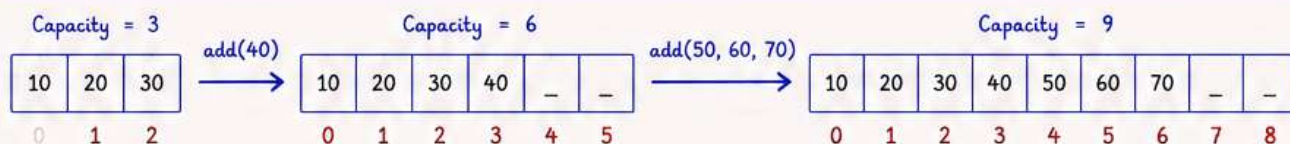
★ When to Use ArrayList ?

- When you need **fast access** to elements by index.
- When insertions and deletions are few compared to **reads**.
- When you need an **ordered** collection that allows **duplicates**.

★ Example

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);
list.add(1, 15); // insert 15 at index 1
System.out.println(list); // [10, 15, 20, 30]
System.out.println(list.get(2)); // 20
```

★ Resizing in ArrayList



Advantages

- ✓ Fast random access.
- ✓ Automatically resizable.
- ✓ Works well for read-intensive operations.
- ✓ Allows duplicate and null elements.

Disadvantages

- ✗ Slow insertions/deletions in middle (shifting required).
- ✗ Extra memory used during resizing.
- ✗ Not thread-safe.



4. LINKEDLIST



★ What is LinkedList ?

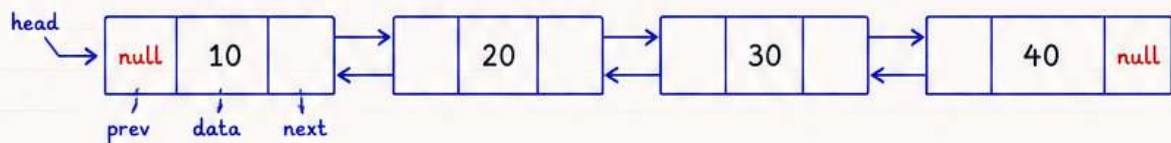
LinkedList is a part of the `List` interface.
It is implemented using a doubly linked list.
It allows insertion and deletion of elements at both ends and in between.

Key Features

- Maintains insertion order.
- Allows duplicate and null elements.
- Good performance for frequent insertions/deletions.
- Slower random access (not index based).
- Can be used as `List`, `Queue` and `Deque`.

★ Internal Working

LinkedList is a doubly linked list where each element (node) contains reference to both previous and next node.



★ Common Methods

<code>add(E e)</code>	Adds element at the end.
<code>addFirst(E e)</code>	Adds element at the beginning.
<code>addLast(E e)</code>	Adds element at the end.
<code>removeFirst()</code>	Removes and returns first element.
<code>removeLast()</code>	Removes and returns last element.
<code>get(int index)</code>	Returns element at index (slow).
<code>set(int index, E e)</code>	Updates element at index.
<code>contains(Object o)</code>	Checks presence of element.
<code>size()</code>	Returns number of elements.

★ LinkedList vs ArrayList

Feature	ArrayList	LinkedList
Data Structure	Resizable Array	Doubly Linked List
Access	Fast (index based)	Slow (index access)
Insert/Delete	Slow in middle $O(n)$	Fast $O(1)$
Memory Usage	Less	More (extra pointers)
Cache Performance	Better	Poorer
Use Case	More read operations	More insert/delete operations

★ Time Complexity

Operation	Complexity
<code>addFirst(), addLast()</code>	$O(1)$
<code>add(index, E e)</code>	$O(n)$
<code>removeFirst(), removeLast()</code>	$O(1)$
<code>remove(index)</code>	$O(n)$
<code>get(index)</code>	$O(n)$
<code>set(index, E e)</code>	$O(n)$
<code>contains(Object o)</code>	$O(n)$
<code>size()</code>	$O(1)$

★ Example

```
LinkedList<String> list = new LinkedList<>();  
list.add("Aman");  
list.add("Rahul");  
list.add("Neha");  
list.addFirst("Start"); // [Start, Aman, Rahul, Neha]  
list.addLast("End");    // [Start, Aman, Rahul, Neha, End]  
list.removeFirst();     // removes Start  
list.removeLast();      // removes End  
System.out.println(list); // [Aman, Rahul, Neha]  
System.out.println(list.get(1)); // Rahul
```

★ When to Use LinkedList ?

- When you need frequent insertions and deletions at start, end or middle.
- When you need to implement stack, queue or deque.
- When memory is not a major constraint.

★ Advantages

- ✓ Fast insertions and deletions.
- ✓ Can be used as `List`, `Queue` and `Deque`.
- ✓ Maintains insertion order.

★ Disadvantages

- ✗ Slow random access.
- ✗ More memory usage.
- ✗ Poor cache performance.



5. SET INTERFACE



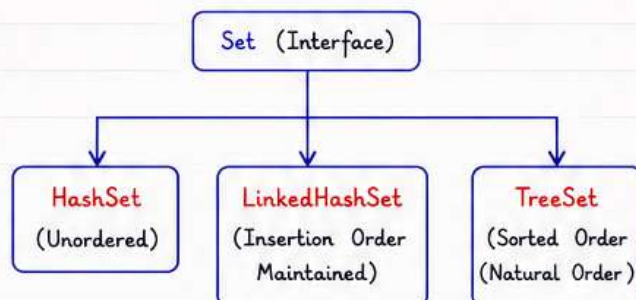
★ What is Set ?

Set is a collection that **does not allow duplicate** elements. It is used to store **unique** elements only.

Key Features

- Does not allow duplicate elements.
- At most one **null** element allowed (depends on implementation).
- Not index based.
- Used to store unique elements.
- Performance depends on the implementation.

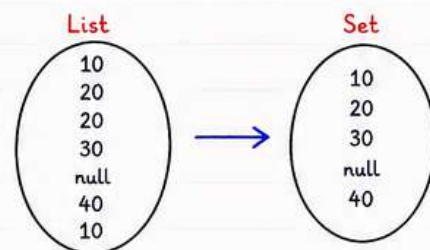
★ Set Implementations



★ Example

Input List : [10, 20, 20, 30, null, 40, 10]

Set (HashSet) : {10, 20, 30, null, 40}



★ Comparison of Set Implementations

Feature	HashSet	LinkedHashSet	TreeSet
Order	No Order	Insertion Order Maintained	Sorted Order (Natural Order)
Duplicates	Not Allowed	Not Allowed	Not Allowed
Null Element	1 Null Allowed	1 Null Allowed	Not Allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance	Very Fast	Fast	Slower ($O(\log n)$)
Use Case	When order is not important	When insertion order is required	When sorted order is required

★ Common Methods

- `add(E e)` → Adds element
- `remove(Object o)` → Removes element
- `contains(Object o)` → Checks if element exists
- `size()` → Returns size
- `isEmpty()` → Checks if empty
- `clear()` → Removes all elements
- `iterator()` → Returns an iterator

★ Example (HashSet)

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(20); // duplicate, will not be added
set.add(null); // one null allowed
set.add(30);
set.add(10); // duplicate, will not be added
System.out.println(set); // Output: [null, 20, 10, 30]
```



Key Takeaway

- Use **HashSet** when you need fast performance and order is not important.
- Use **LinkedHashSet** when you need unique elements and insertion order.
- Use **TreeSet** when you need unique elements in sorted order.



6. MAP INTERFACE



★ What is Map ?

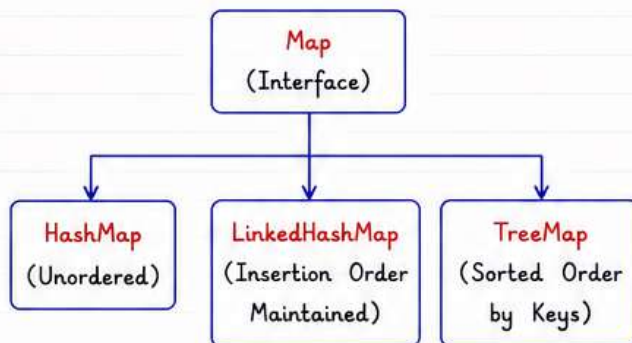
Map is an interface in Java Collections that stores data in **key-value** pairs.

Keys are **unique**, but values can be duplicate. Each key maps to exactly one value.

Key Features

- Stores data in **key-value** pairs.
- Keys are **unique**, values can be duplicate.
- Each key maps to **one** value.
- At most one **null** key allowed (in some implementations).
- Multiple **null** values allowed.
- Not thread-safe (depends on implementation).

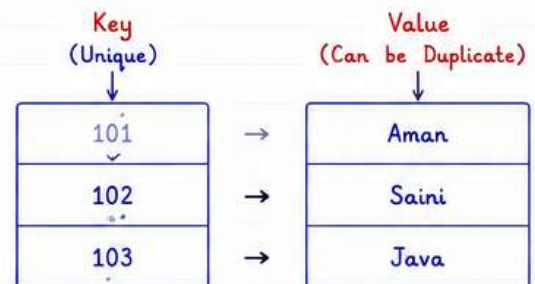
★ Map Implementations



★ Example

```

Map<Integer, String> map = new HashMap<>();
map.put(101, "Aman");
map.put(102, "Saini");
map.put(103, "Java");
System.out.println(map);
// Output: {101=Aman, 102=Saini, 103=Java}
  
```



★ Common Methods

- | | |
|-------------------------------|-------------------------------------|
| • put(K key, V value) | → Inserts or updates entry |
| • get(Object key) | → Returns value for the key |
| • remove(Object key) | → Removes entry for the key |
| • containsKey(Object key) | → Checks if key exists |
| • containsValue(Object value) | → Checks if value exists |
| • size() | → Returns number of key-value pairs |
| • isEmpty() | → Checks if map is empty |
| • clear() | → Removes all entries |
| • keySet() | → Returns set of keys |
| • values() | → Returns collection of values |
| • entrySet() | → Returns set of key-value pairs |

★ Comparison of Map Implementations

Feature	HashMap	LinkedHashMap	TreeMap
Order	No Order	Insertion Order Maintained	Sorted Order (by Keys)
Null Key	1 Null Key Allowed	1 Null Key Allowed	Not Allowed
Null Values	Multiple Nulls Allowed	Multiple Nulls Allowed	Multiple Nulls Allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance	Very Fast	Fast	Slower ($O(\log n)$)
Use Case	When order is not required	When insertion order is required	When sorted order of keys is required

★ When to Use Map ?

- ✓ Use Map when you need to store and access elements using a unique key.
- ✓ Use HashMap for best performance when order is not important.
- ✓ Use LinkedHashMap when you need insertion order.
- ✓ Use TreeMap when you need sorted keys.

Think of **Map** as a Dictionary where each word (key) has a meaning (value).

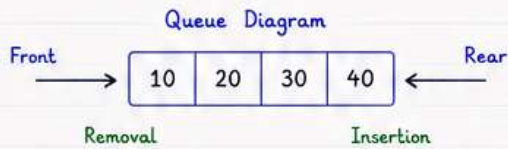




7. QUEUE, PRIORITYQUEUE & STACK

★ 1. QUEUE (Interface)

Queue follows FIFO (First In First Out) order. Elements are added at the rear and removed from the front.



Common Methods

- `offer(E e)` → Inserts element
- `poll()` → Retrieves and removes head
- `peek()` → Retrieves head without removing
- `isEmpty()` → Checks if queue is empty
- `size()` → Returns number of elements

Implementations

- `LinkedList` → Most common Queue implementation
- `ArrayDeque` → Better performance, resizable array (recommended)
- `PriorityQueue` → Elements ordered by priority

Example (Using LinkedList)

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(10);    // add
queue.offer(20);
queue.offer(30);
System.out.println(queue.peek()); // 10
System.out.println(queue.poll()); // 10 (removed)
System.out.println(queue);        // [20, 30]
```

★ 2. PRIORITYQUEUE

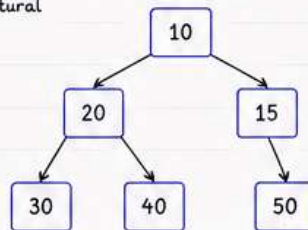
PriorityQueue orders elements based on their natural ordering or a custom Comparator.

The head is the smallest element by default (Min-Heap).

Example

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10); pq.offer(20); pq.offer(40);
System.out.println(pq); // [10, 30, 20, 40] (order may vary)
System.out.println(pq.poll()); // 10 (smallest element)
```

Min-Heap Diagram



Common Methods

- `offer(E e)` → Inserts element
- `poll()` → Retrieves & removes head (highest priority)
- `peek()` → Retrieves head without removing
- `size()` → Returns size
- `isEmpty()` → Checks if empty

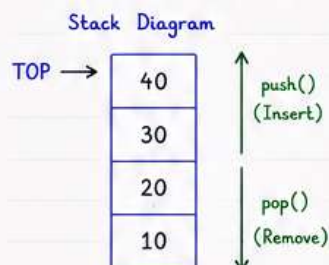


Note : PriorityQueue does NOT maintain insertion order. Elements are stored based on priority (natural order or comparator).

★ 3. STACK

Stack follows LIFO (Last In First Out) order.

The last element inserted is the first one removed.



Common Methods

- `push(E e)` → Inserts element
- `pop()` → Removes top element
- `peek()` → Retrieves top without removing
- `isEmpty()` → Checks if empty
- `search(E e)` → Returns position of element from top

Example (Using Deque / ArrayDeque)

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.peek()); // 30
System.out.println(stack.pop()); // 30
System.out.println(stack);        // [20, 10]
```

Note :

- Stack class is legacy. Prefer Deque (ArrayDeque).
- Deque can be used as Stack or Queue.

★ Quick Comparison

Feature	Queue	PriorityQueue	Stack
Order	FIFO (First In First Out)	Based on Priority (Min-Heap by default)	LIFO (Last In First Out)
Insertion	At Rear	Anywhere	At Top
Deletion	From Front	From Head (Highest Priority)	From Top
Implementation	LinkedList / ArrayDeque	PriorityQueue	Deque (ArrayDeque)
Use Case	Task scheduling, BFS	Task scheduling by priority, Dijkstra's Algorithm	Backtracking, Undo operation, Expression evaluation



8. HASHMAP vs LINKEDHASHMAP vs TREEMAP

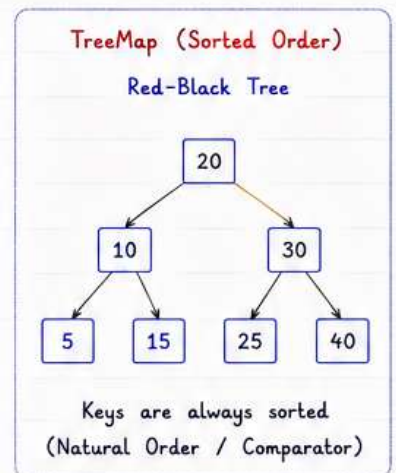
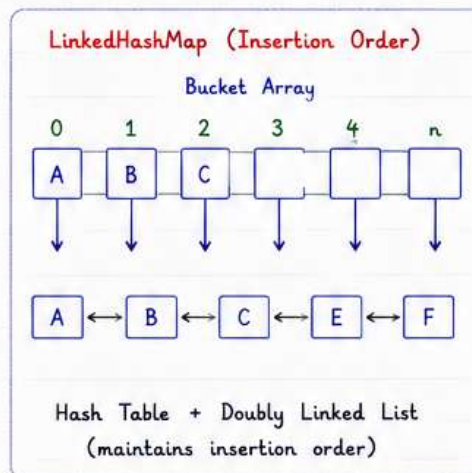
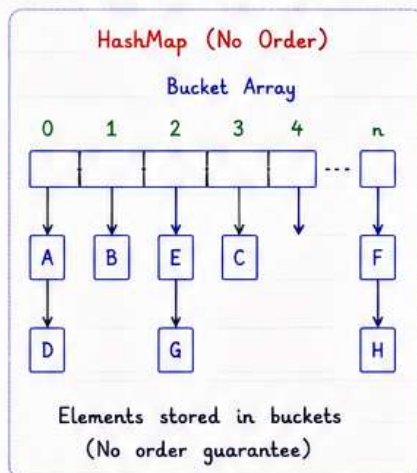


★ Overview

All three classes implement `Map` interface and store data in key-value pairs, but they differ in ordering, null handling and internal structure.

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order (Random)	Insertion order maintained	Sorted order by keys (Natural order or Comparator)
Null Key	1 null key allowed	1 null key allowed	No null key
Null Values	Multiple null values allowed	Multiple null values allowed	Multiple null values allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance (get/put)	Very fast (average $O(1)$)	Fast (average $O(1)$)	Slower ($O(\log n)$)
Use Case	When order is not important and we need best performance	When insertion order needs to be maintained (e.g. caching, LRU)	When keys need to be sorted (range operations, navigable map)

★ Internal Structure (Visual)



★ Example (Same Input)

Input Insert Order : 30, 10, 20, 40, 5		
HashMap (No Order) {30=Thirty, 10=Ten, 20=Twenty, 40=Forty, 5=Five} (Order not guaranteed)	LinkedHashMap (Insertion Order) {30=Thirty, 10=Ten, 20=Twenty, 40=Forty, 5=Five} (Insertion order maintained)	TreeMap (Sorted Order by Keys) {5=Five, 10=Ten, 20=Twenty, 30=Thirty, 40=Forty} (Sorted by keys)

★ Key Points

- **HashMap** is fastest.
- **LinkedHashMap** maintains insertion order with slight extra memory.
- **TreeMap** keeps keys sorted and is useful for range queries (`firstKey()`, `lastKey()`, `subMap()`).
- Choose collection based on order requirement & performance.

★ Quick Summary

Use HashMap when :	Use LinkedHashMap when :	Use TreeMap when :
✓ Order is not important ✓ Fastest performance needed ✓ Default choice for most cases	✓ You need insertion order ✓ Useful in caching (LRU Cache) ✓ Slightly more memory than HashMap	✓ You need sorted keys ✓ Range / navigation operations ✓ Slower but sorted data

COMMENT PDF

— TO GET —

COMPLETE PDF

OF

COLLECTION

JAVA COLLECTIONS FRAMEWORK

TYPES OF COLLECTIONS

-  LIST
-  SET
-  MAP
-  QUEUE
-  STACK
-  COMPARATOR & COMPARABLE

WHY COLLECTIONS ?

-  EFFICIENT DATA HANDLING
-  REUSABILITY
-  BETTER PERFORMANCE
-  DATA STRUCTURE SUPPORT
-  INTEROPERABILITY
-  SCALABILITY



SAVE



SHARE



FOLLOW



aman_views



≡ JAVA EXCEPTION HANDLING ≡

CHAPTER 1

Page 1/6

1. WHAT IS EXCEPTION ?

An **exception** is an **event** that occurs during the execution of a program that **disrupts** the normal flow of instructions.

In Java, **exceptions** are **objects** that represent errors or unexpected situations.

Examples of Exceptions

- ★ Division by zero
- ★ File not found
- ★ Null pointer access
- ★ Array index out of range
- ★ Invalid user input

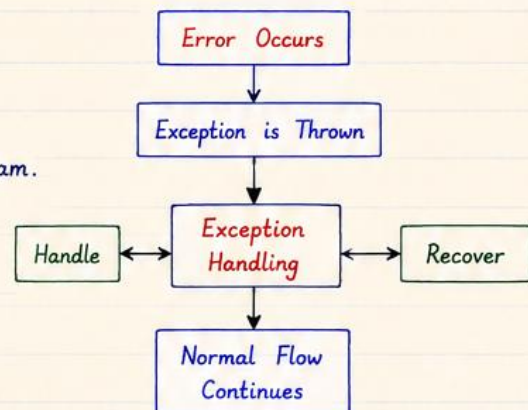
Exception Handling is the mechanism to handle runtime errors so that the **normal flow** of the program can be **maintained**.

Note :

Exceptions are checked at runtime (not at compile time).

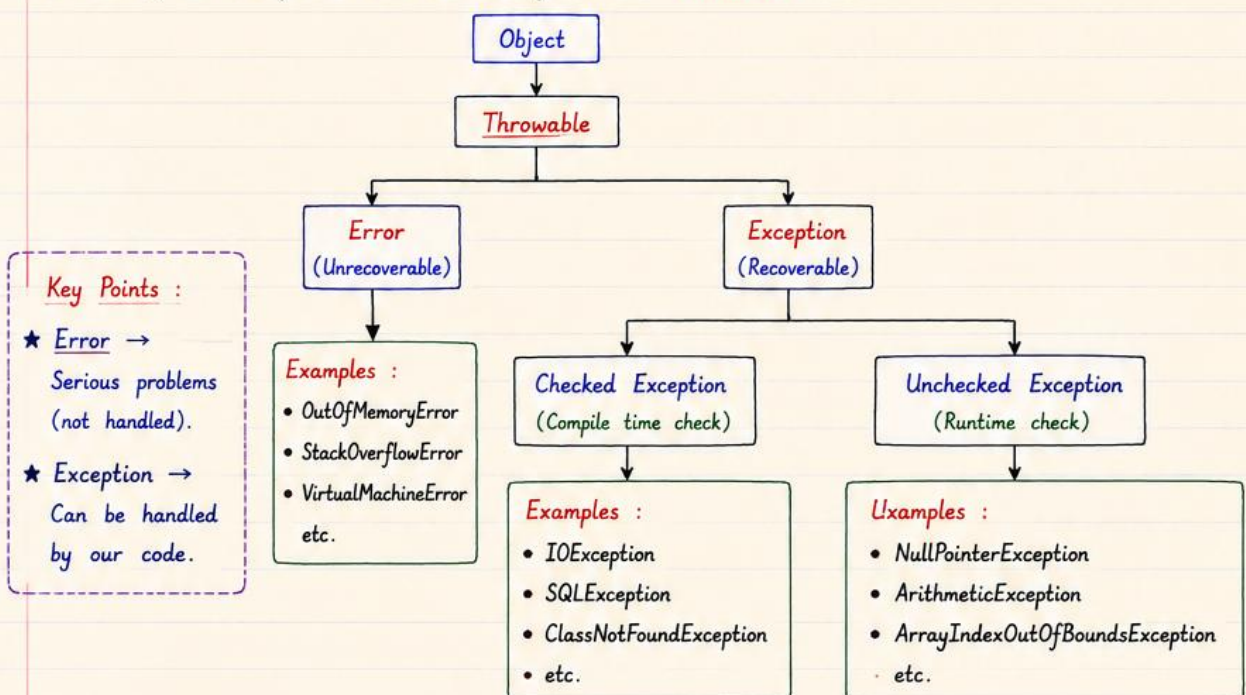
2. WHY EXCEPTION HANDLING ?

- ★ Maintains normal flow of the application.
- ★ Avoids abnormal termination of the program.
- ★ Helps to find and fix bugs easily.
- ★ Provides better user experience.
- ★ Improves code reliability and robustness.



3. EXCEPTION HIERARCHY IN JAVA

In Java, all exceptions are derived from the **Throwable** class.



Key Points :

- ★ **Error** → Serious problems (not handled).
- ★ **Exception** → Can be handled by our code.

JAVA EXCEPTION HANDLING

CHAPTER 1

Page 2/6

★ Types of Exceptions

In Java, exceptions are mainly of two types.

1. Checked Exceptions - Checked at compile time.
2. Unchecked Exceptions - Checked at runtime.

Checked Exceptions	Unchecked Exceptions
• Checked at compile time • Must be handled or declared using throws • Subclasses of Exception • Example: IOException FileNotFoundException	• Checked at runtime • Not required to handle • Subclasses of RuntimeException • Example: NullPointerException ArithmeticException

★ Common Built-in Exceptions

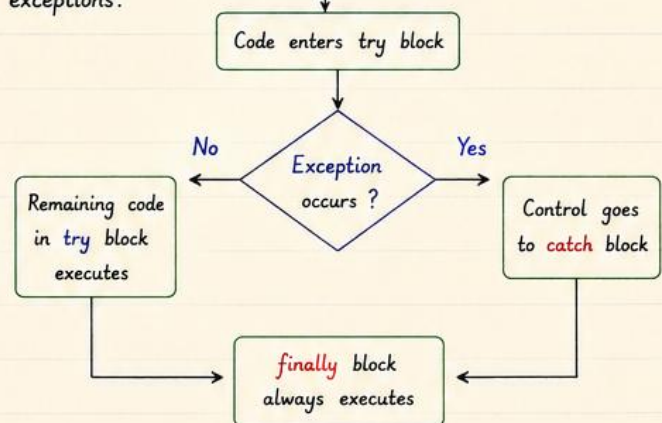
Exception	Description
ArithmeticException	Occurs when an exceptional arithmetic condition occurs (e.g., divide by zero)
NullPointerException	Occurs when an object reference is null
ArrayIndexOutOfBoundsException	Occurs when accessing invalid index in an array
NumberFormatException	Occurs when converting invalid String to number
IOException	Occurs when an input-output operation fails
ClassNotFoundException	Occurs when a class is not found

★ try, catch, finally

We use **try**, **catch** and **finally** block to handle exceptions.

```
try {
    // code that may cause exception
    int a = 10, b = 0;
    int c = a / b; // may throw ArithmeticException
    System.out.println(c);
}
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
finally {
    System.out.println("Finally block executed");
}
```

Flow of try-catch-finally



★ throw and throws

throw

- Used to explicitly throw an exception.
- Used inside the method.
- Syntax:

```
throw exceptionObject;
```

Example:

```
if (age < 18) {
    throw new ArithmeticException("Age must be 18 or above");
}
```

throws

- Used to declare an exception.
- Used in method signature.
- Indicates that the method may throw an exception.
- Syntax:

```
returnType methodName() throws ExceptionType;
```

Example:

```
int divide(int a, int b) throws ArithmeticException {
    if (b == 0)
        throw new ArithmeticException("Cannot divide by zero");
    return a / b;
}
```

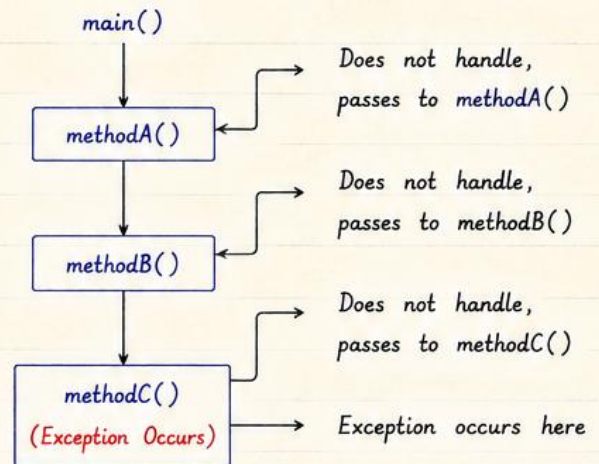
★ Exception Propagation

If an exception is not handled in a method, it is passed to the calling method.

This continues up the call stack until the exception is handled.

Note :

If no method handles the exception, the program terminates abnormally and prints exception message.



★ Custom Exception

We can create our own exceptions by extending **Exception** class.

```

class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}
// user defined
// exception
    
```

★ Multiple catch Blocks

A try block can have multiple catch blocks to handle different types of exceptions.

```

try {
    int a = 10, b = 0;
    int c = a / b; // may throw ArithmeticException
    int arr[] = new int[2];
    arr[5] = 10; // may throw ArrayIndexOutOfBoundsException
}
catch (ArithmeticException e) {
    System.out.println("Arithmetic Exception");
}
catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array Index Exception");
}
catch (Exception e) {
    System.out.println("Some other Exception");
}
    
```

Order of catch blocks

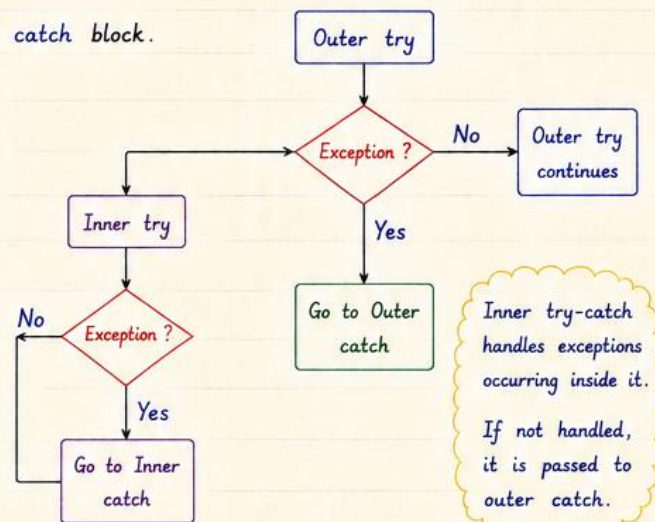
- Specific exceptions must be written before general exceptions.
- General catch (Exception e) should be last.
- If general catch is written first, compiler will give error.

★ Nested try Block

A try block can be placed inside another try or catch block.

```

try {
    int a[] = new int[2];
    a[5] = 30; // may throw ArrayIndexOutOfBoundsException
    try {
        int x = 10 / 0; // may throw ArithmeticException
    }
    catch (ArithmeticException e) {
        System.out.println("Inner try-catch");
    }
    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Outer try-catch");
    }
}
    
```



MULTITHREADING IN JAVA

CHAPTER 1

Page 4/6

B. MULTITHREADING

★ What is Thread ?

A thread is the **smallest unit of execution** in a program.

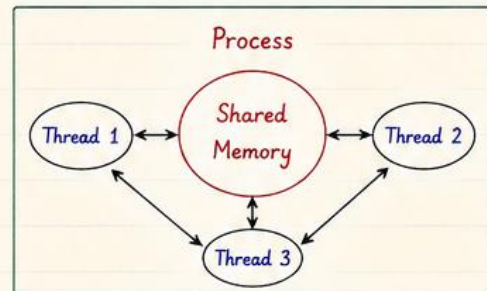
A program can have **multiple threads** running concurrently to perform tasks.

Why Multithreading ?

- Improves performance.
- Better resource utilization.
- Enables concurrent execution.
- Keeps application responsive.
- Essential for modern applications.

★ Benefits of Multithreading

- **Responsiveness** - UI remains responsive.
- **Performance** - Tasks run in parallel.
- **Resource sharing** - Threads share memory.
- **Economy** - Less memory & context switching.
- **Scalability** - Handles more tasks simultaneously.



★ Creating a Thread

There are two ways to create a thread in Java.

1. By extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Test {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();    // starts the thread
    }
}
```

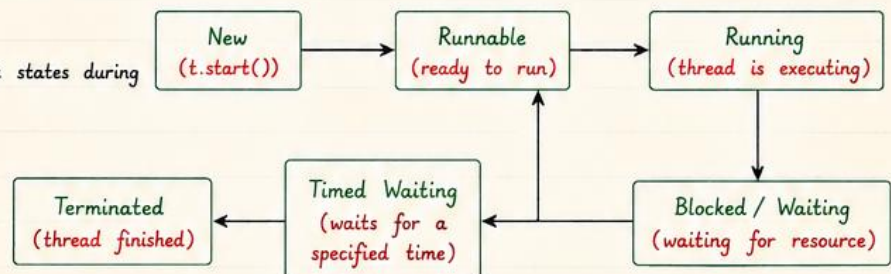
2. By implementing Runnable interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Test {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable();
        Thread t = new Thread(r);
        t.start();    // starts the thread
    }
}
```

★ Thread Life Cycle

A thread goes through different states during its life cycle.



★ start() vs run()

Feature	start()	run()
Who starts	JVM starts a new thread	Called like a normal method
New thread created	Yes	No
Executes in parallel	Yes	No
Calls	start() internally calls run()	Directly executes run()
Example	t.start();	t.run();

MULTITHREADING IN JAVA

CHAPTER 1

Page 5/6

★ Thread Methods

Method	Description
start()	Starts the execution of thread. It internally calls run().
run()	Contains the code to be executed by the thread.
sleep(long millis)	Pauses the current thread for specified time.
join()	Waits for this thread to die.
yield()	Causes the currently executing thread to pause & allow other threads of same priority to execute.
isAlive()	Tests if the thread is alive.

★ Example : Using sleep() and join()

```
class SleepJoinExample extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Thread running : " + i);
            try {
                Thread.sleep(500); // sleep for 0.5 sec
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Test {
    public static void main(String[] args) throws Exception {
        SleepJoinExample t = new SleepJoinExample();
        t.start();
        t.join(); // main thread waits for t to finish
        System.out.println("Thread has finished");
    }
}
```

Output

```
Thread running : 1
Thread running : 2
Thread running : 3
Thread running : 4
Thread running : 5
Thread has finished
```

Note :

join() makes the main thread wait until the child thread completes.

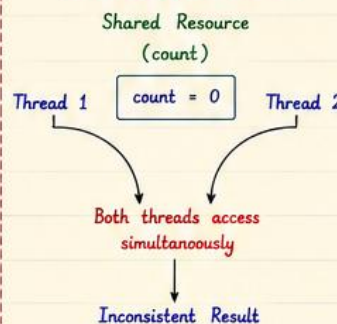
★ Synchronization (synchronized)

Synchronization is used to control access to shared resources to avoid inconsistent data.

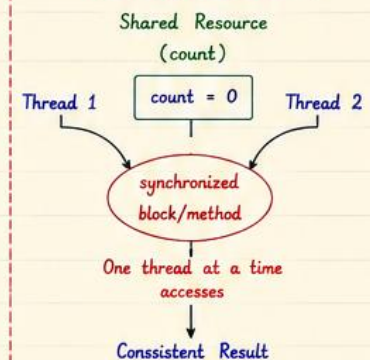
```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

Without Synchronization



With Synchronization



★ Daemon Thread

A daemon thread is a low priority thread that runs in background to perform tasks like garbage collection.

```
Thread t = new Thread(new MyTask());
t.setDaemon(true); // set as daemon thread
t.start();
```

MULTITHREADING IN JAVA

★ Thread Priorities

A thread priority helps to determine the order in which threads are scheduled for execution.

Java supports 3 priority levels.

Priority Constant	Value	Description
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Normal priority (default)
Thread.MAX_PRIORITY	10	Highest priority

```
Thread t1 = new Thread();
```

```
Thread t2 = new Thread();
```

```
t1.setPriority(Thread.MIN_PRIORITY); // 1
```

```
t2.setPriority(Thread.MAX_PRIORITY); // 10
```

```
System.out.println("t1 priority: " + t1.getPriority());
```

```
System.out.println("t2 priority: " + t2.getPriority());
```

★ Inter Thread Communication

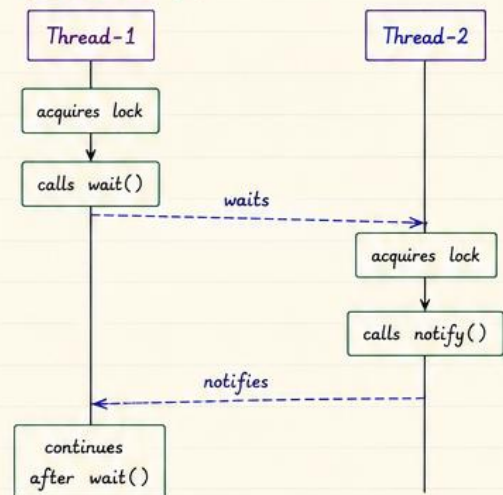
Threads can communicate with each other using `wait()`, `notify()` and `notifyAll()` methods of `Object` class.

```
class Communication {  
    public static void main(String[] args) {  
        final Object lock = new Object();  
  
        Thread t1 = new Thread(() -> {  
            synchronized (lock) {  
                System.out.println("Thread-1 : waiting...");  
                try { lock.wait(); } catch (InterruptedException e) {}  
                System.out.println("Thread-1 : notified!");  
            }  
        });  
  
        Thread t2 = new Thread(() -> {  
            synchronized (lock) {  
                try { Thread.sleep(1000); } catch (InterruptedException e) {}  
                System.out.println("Thread-2 : notifying...");  
                lock.notify();  
            }  
        });  
        t1.start();  
        t2.start();  
    }  
}
```

Note :

`wait()`, `notify()` and `notifyAll()` must be called from synchronized block or method.

How it works ?

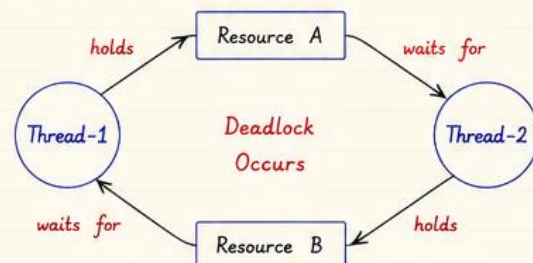


★ Deadlock

Deadlock occurs when two or more threads are blocked forever, each waiting for a resource held by the other.

Example Scenario

Thread-1 holds Resource A and waits for B
Thread-2 holds Resource B and waits for A
Both threads are waiting for each other forever.



★ Important Points

- Always start a thread using `start()` method, not `run()`.
- Use synchronization to avoid data inconsistency.
- Use proper priority and communication to control threads.
- Avoid deadlock by acquiring locks in fixed order.
- Daemon thread does not prevent JVM from exiting.

```
Thread t = new Thread(new MyTask());  
t.setName("Worker-1");  
t.setDaemon(true); // daemon thread  
t.setPriority(Thread.NORM_PRIORITY);  
t.start();
```

COMMENT PDF

— TO GET —

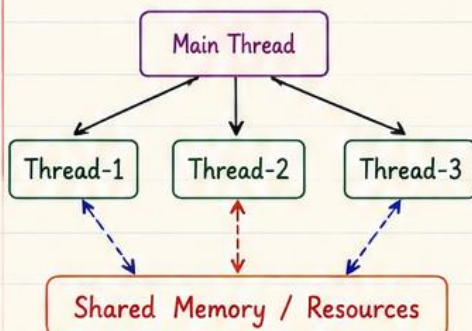
COMPLETE PDF

— OF —

MULTITHREADING AND EXCEPTION HANDLING

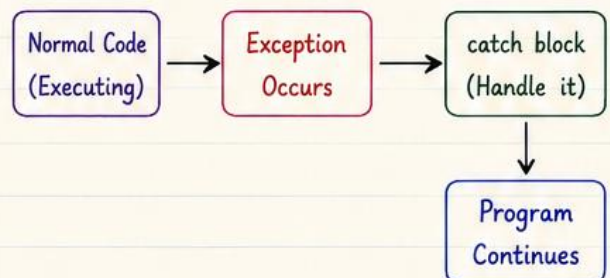
MULTITHREADING

- ★ Multiple threads execute concurrently.
- ★ Threads share the same memory.
- ★ Improves performance and responsiveness.



EXCEPTION HANDLING

- ★ Exception is an event that disrupts normal flow.
- ★ Handled using try, catch, finally, throw, throws.
- ★ Keeps the program stable and error-free.



SAVE



SHARE



FOLLOW



code_with_aman



Java Abstraction & Interface

Handwritten Notes / Handbook

Best Notes on Interface & Abstraction

Zero To Pro - From Beginner To Advanced (Everything Covered)

1. UNDERSTANDING ABSTRACTION

Abstraction is the process of hiding the implementation details and showing only the functionality to the user.

→ It helps us to focus on what an object does instead of how it does it.

Key Point :

Abstraction = Hide the implementation
Show the functionality

2. WHY DO WE NEED ABSTRACTION ?

- Reduces complexity.
- Increases security (internal details are hidden).
- Improves maintainability.
- Helps in achieving loose coupling.
- Promotes reusability and scalability.

Real World Examples :

Car



We drive the car but don't know how engine works.

ATM



We use buttons but don't know internal processing.

Mobile Phone



We use apps but don't know how OS works.

3. PARTIAL VS COMPLETE ABSTRACTION

Partial Abstraction	Complete Abstraction
• Some details are visible to the user. • Achieved using Abstract Class.	• Only essential features are visible. • Achieved using Interface.

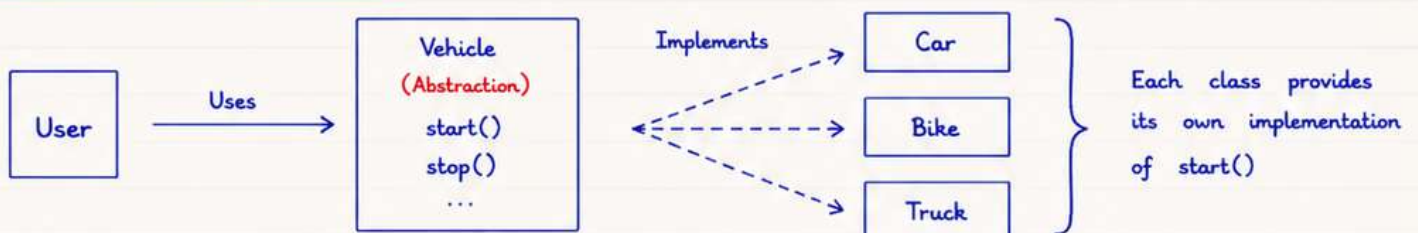
4. SIMPLE EXAMPLE IN JAVA

```
abstract class Vehicle {  
    abstract void start(); // abstract method (no body)  
    void stop() {  
        System.out.println("Vehicle stopped");  
    }  
}
```

→ start() is an abstract method. (Implementation is hidden)

→ stop() is a concrete method. (Implementation is provided)

5. ABSTRACTION VISUALIZED

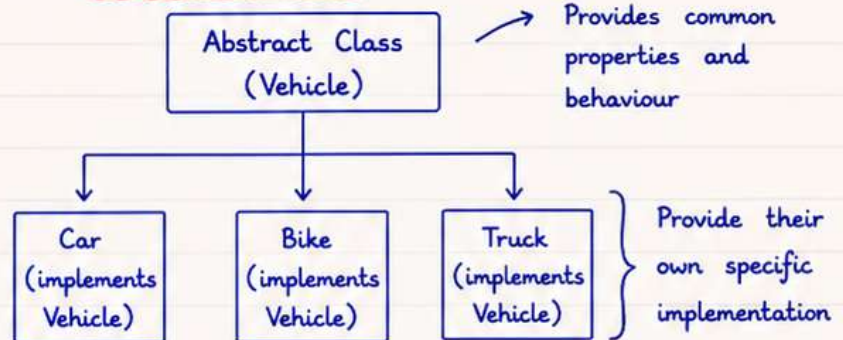


Java Abstraction and Interface

1. ABSTRACT CLASS - DEEP DIVE

An abstract class is a class that cannot be instantiated and is meant to be inherited by other classes.

How it works ?



2. KEY FEATURES OF ABSTRACT CLASS

- It can have abstract and non-abstract methods.
- It can have constructors.
- It can have instance variables (state).
- It can have static methods.
- It cannot be instantiated.
- It is extended using "extends" keyword.
- A subclass must provide implementation for all abstract methods.

WHAT IS ALLOWED IN ABSTRACT CLASS ?

Feature	Allowed ?
Abstract Methods	✓
Concrete Methods	✓
Constructors	✓
Instance Variables	✓
Static Methods	✓
Final Methods	✓
Object Creation	✗

3. EXAMPLE IN JAVA

```
abstract class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    abstract void start(); // abstract method

    void stop() {
        System.out.println("Vehicle Stopped");
    }
}
```

- Vehicle is an abstract class.
- It has an abstract method start() without implementation.
- It has a concrete method stop() with implementation.
- Constructor is used to initialize state (brand).

4. IMPORTANT NOTES

- We cannot create object of an abstract class.
- If a class extends an abstract class, it must implement all abstract methods or itself be declared abstract.
- Abstract class helps in providing a common base and partial implementation.

Java Abstraction and Interface

1. INTERFACE - DEEP DIVE

An interface is a blueprint of a class. It defines a contract that classes must follow.

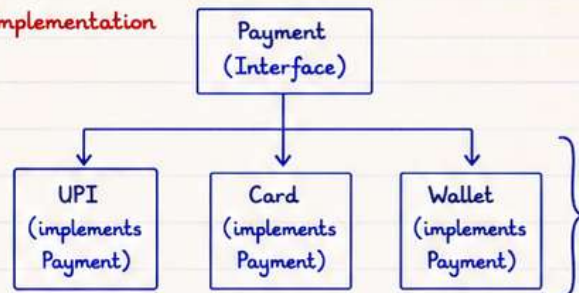
Why Interface ?

- Achieve 100% abstraction.
- Multiple inheritance is possible.
- Loose coupling and flexibility.
- Used for defining a standard behavior.

2. INTERFACE STRUCTURE

```
interface Payment {  
    void pay();    // abstract method  
    void refund(); // abstract method  
}
```

Implementation



Multiple classes can implement the same interface.

3. TYPES OF METHODS IN INTERFACE (Modern Java)

Type	Syntax	Can have body ?	From Version	Example
Abstract Method	<code>void show();</code>	No (by default)	All Versions	<code>void show();</code>
Default Method	<code>default void show() { ... }</code>	Yes	Java 8	<code>default void show() { System.out.println("Hi"); }</code>
Static Method	<code>static void show() { ... }</code>	Yes	Java 8	<code>static void show() { System.out.println("Hi"); }</code>
Private Method	<code>private void show() { ... }</code>	Yes (Only inside interface)	Java 9	<code>private void show() { // helper method }</code>

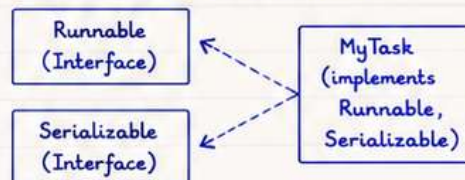
4. CONSTANTS IN INTERFACE

- All variables in interface are public static final by default.
- They must be initialized.

```
interface Config {  
    int MAX_LIMIT = 100;    // public static final  
    String APP_NAME = "MyApp";  
}
```

5. MULTIPLE INHERITANCE

- A class can implement multiple interfaces.
- This is not possible with classes.



6. EXAMPLE - INTERFACE IN REAL WORLD

```
interface Notification {  
    void send(String msg);  
    default void printHeader() {  
        System.out.println("--- Notification ---");  
    }  
}
```

Implementation



```
class EmailNotification implements Notification {  
    public void send(String msg) {  
        System.out.println("Email : " + msg);  
    }  
}  
  
class SMSNotification implements Notification {  
    public void send(String msg) {  
        System.out.println("SMS : " + msg);  
    }  
}
```

Use



Client
uses
Notification
interface

- ✓ Loose coupling
- ✓ Easy to extend
- ✓ Follows contract

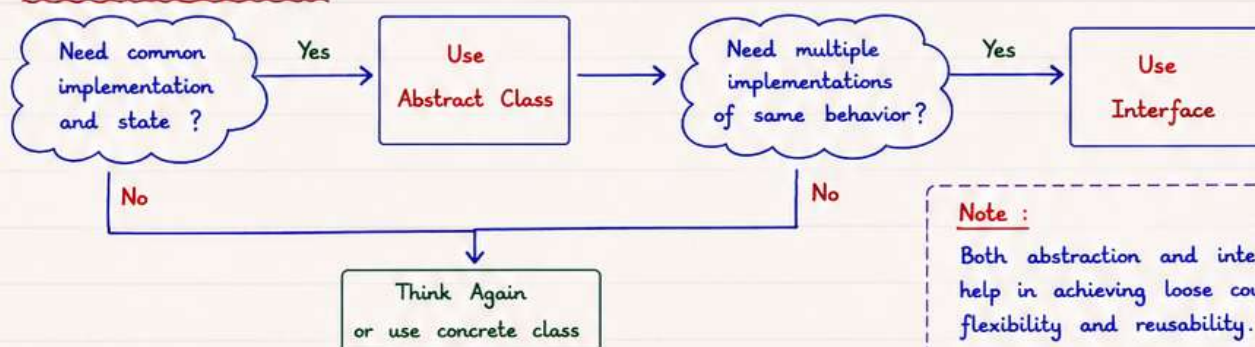
Java Abstraction and Interface

4. ABSTRACT CLASS vs INTERFACE

Complete Comparison

Feature	Abstract Class	Interface
1. Declaration Keyword	abstract keyword	interface keyword
2. Methods	Can have abstract and concrete methods.	Can have only abstract methods (till Java 7). From Java 8 → all types.
3. Method Body	Abstract methods have no body. Concrete methods have body.	Abstract methods have no body. (default/static methods can have body)
4. Default Methods	✗	Yes (Java 8 and above)
5. Static Methods	Can have static methods.	Yes (Java 8 and above)
6. Private Methods	Can have private methods.	Yes (Java 9 and above)
7. Variables (Instance)	Can have instance variables.	Cannot have instance variables.
8. Variables (Static)	Can have static variables.	Can have static final variables (acts as constant).
9. Constructors	Can have constructors.	Cannot have constructors.
10. Multiple Inheritance	No (A class can extend only one class.)	Yes (A class can implement multiple interfaces.)
11. Object Creation	Cannot create object.	Cannot create object.
12. Access Modifiers	Can use any access modifier.	By default → public (methods are public)
13. State (Data)	Can maintain state.	Cannot maintain state.
14. Extends / Implements	Use extends keyword.	Use implements keyword.
15. Performance	Better (slightly faster)	Slightly lower (negligible)
16. Use Case	To provide common implementation and state.	To define a contract (behavior).
17. When to use ?	When you want to provide common code and maintain state.	When you want to define behavior that multiple classes must follow.
18. Example	<pre>abstract class Vehicle { abstract void start(); void stop() { ... } }</pre>	<pre>interface Payment { void pay(); default void receipt() { ... } }</pre>

Quick Decision Guide



Note :

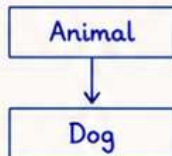
Both abstraction and interface help in achieving loose coupling, flexibility and reusability.

Java Abstraction and Interface

5. IS-A vs HAS-A RELATIONSHIP

1. IS-A Relationship (Inheritance)

In inheritance (IS-A), one class is a type of another class. It represents a parent-child relationship.



Dog IS-A Animal.

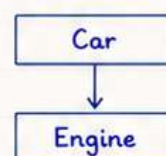
Java Example :

```

class Animal { }
class Dog extends Animal { }
  
```

2. HAS-A Relationship (Composition)

In composition (HAS-A), one class contains an object of another class. It represents an owner-component relationship.



Car HAS-A Engine.

Java Example :

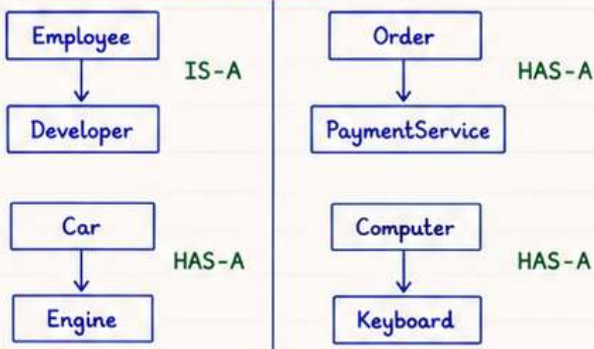
```

class Engine { }
class Car {
    Engine engine = new Engine();
}
  
```

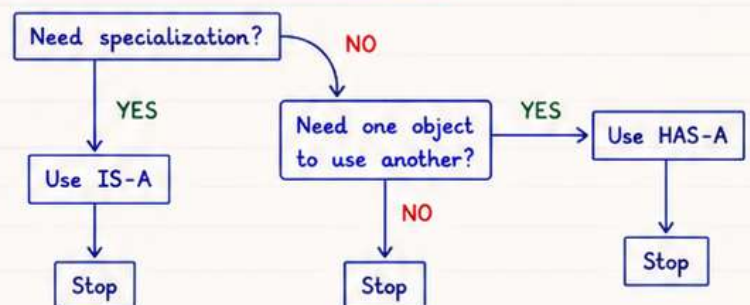
3. IS-A vs HAS-A (Quick Comparison)

Aspect	IS-A (Inheritance)	HAS-A (Composition)
Meaning	One class is a type of another	One class contains another object
Keyword	extends	Object reference
Coupling	Tight Coupling	Loose Coupling
Reuse	Reuse through Inheritance	Reuse through Composition
Relationship	Parent-Child	Owner-Component
Flexibility	Less Flexible	More Flexible
Best Use Case	When there is a true parent-child relationship	When one class uses another class for its functionality

4. Real Project Examples



5. When to Use? (Decision Flow)



6. Best Practice

- Prefer Composition (HAS-A) over Inheritance (IS-A) when possible.
- Use IS-A only when there is a true parent-child relationship.
- Use HAS-A for flexibility and loose coupling.

Java Abstraction and Interface

6. JAVA 8 INTERFACE FEATURES & FUNCTIONAL INTERFACES

1. Why Java 8 changed Interfaces ?

- Earlier interfaces had only abstract methods.
- Java 8 added Default and Static methods.
- This make interfaces more powerful and suitable for real world use.

2. New Features in Interface (Java 8+)

1. Default Methods → has a body, provides default implementation.
2. Static Methods → belongs to interface, not to object.
3. Private Methods → helper methods inside interface (Java 9+).
4. Multiple Inheritance → a class can implement multiple interfaces.

3. Default Method Example

```
interface Vehicle {
    void start();           // abstract method
    default void stop() {   // default method
        System.out.println("Vehicle Stopped");
    }
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car Started");
    }
}
```

Child class
is not forced
to override
default method.

4. Static Method in Interface

```
interface MathUtil {
    static int add(int a, int b) {
        return a + b;
    }
}

// Usage
int res = MathUtil.add(10, 20);
```

Called using
Interface name.
Not through
object.

5. Private Method in Interface (Java 9+)

```
interface MessageService {
    default void send(String msg) {
        if (isValid(msg)) {
            System.out.println("Message Sent");
        }
    }
    private boolean isValid(String msg) {
        return msg != null && !msg.isEmpty();
    }
}
```

Private helper
method used
inside interface
only.

6. Functional Interface

- An interface with exactly one abstract method is called Functional Interface.
- It can be used with lambda expressions (@FunctionalInterface).
- It promotes functional programming in Java.

Functional
Interface

Used with
Lambda Expression

() → {
code
}

7. Built-in Functional Interfaces (Most Used)

Interface	Package	Method	Description	Example
Predicate<T>	java.util.function	boolean test(T t)	Takes input, returns boolean	t → t.length() > 5
Function<T,R>	java.util.function	R apply(T t)	Takes input, returns result	s → s.toUpperCase()
Consumer<T>	java.util.function	void accept(T t)	Takes input, returns nothing	s → System.out.println(s)
Supplier<T>	java.util.function	T get()	Takes nothing, returns result	() → new ArrayList<>()

8. Functional Interface Example

```
@FunctionalInterface
interface MyFunctional {
    void show(String msg); // only one abstract method
}

public class Test {
    public static void main(String[] args) {
        MyFunctional mf = (msg) → System.out.println(msg);
        mf.show("Hello Java 8");
    }
}
```

Lambda
expression
using
functional
interface.

9. Summary

- ✓ Interfaces in Java 8 became more powerful.
- ✓ Default, Static and Private methods improved reusability.
- ✓ Functional Interfaces enable lambda expressions.
- ✓ Java 8 interfaces are the backbone of modern Java.

Remember : Functional Interface + Lambda = Powerful Java 8

Java Abstraction and Interface

7. LAMBDA EXPRESSIONS - COMPLETE GUIDE

1. What is Lambda Expression ?

Lambda Expression is a short way to implement a Functional Interface. It helps in writing clean, concise and readable code.

Syntax :

(parameters) → expression
or
(parameters) → { statements; }

2. Why Lambda is Possible ?

- Functional Interface has only one abstract method.
- JVM knows exactly which method to implement.
- So we can provide implementation using → .

Before Java 8 (using Anonymous Class)

```
Runnable r = new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
};
```

After Java 8 (Lambda)

```
Runnable r = () →
    System.out.println("Hello");
```

3. Lambda Examples

No Parameter	One Parameter	Multiple Parameters	With Multiple Statements
Runnable r = () → System.out.println("Hi");	Consumer<String> c = (name) → System.out.println(name);	BiFunction<Integer,Integer,Integer> add = (a, b) → a + b;	Comparator<Integer> comp = (a, b) → { if(a > b) return 1; else if(a < b) return -1; else return 0; };

4. Lambda with Functional Interface

Functional Interface	Method	Lambda Example	Use Case
Predicate<T>	boolean test(T t)	x → x % 2 == 0	Check condition
Function<T,R>	R apply(T t)	s → s.length()	Transform input to output
Consumer<T>	void accept(T t)	x → System.out.println(x)	Consume input
Supplier<T>	T get()	() → new ArrayList<>()	Provide / Supply data

5. Method Reference (::)

- Another way to write lambda.

Lambda	Method Reference
s → s.toUpperCase()	String::toUpperCase
System.out::println	System.out::println
() → Math.random()	Math::random
list → list.size()	List::size

6. Before vs After Java 8

Before Java 8 (Verbose Code)

```
List<String> list = Arrays.asList("A","B","C");
List<String> upper = new ArrayList<>();
for(String s : list) {
    upper.add(s.toUpperCase());
}
System.out.println(upper);
```

After Java 8 (Lambda + Stream)

```
List<String> list = Arrays.asList("A","B","C");
List<String> upper = list.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
System.out.println(upper);
```

7. Where Lambda is Used ?

- ✓ Collection Framework (forEach, sort, filter)
- ✓ Stream API (map, filter, collect)
- ✓ Event Handling (GUI, Web)
- ✓ Multithreading (Runnable, Callable)
- ✓ Functional Programming in Java

8. Summary

- Lambda makes code short, clean and easy to read.
- Works only with Functional Interface.
- Reduces boilerplate code (Anonymous class).
- Enables Functional Programming in Java.
- Widely used in real world (Streams, Collections, APIs).

Remember : Functional Interface + Lambda = Modern Java (Clean, Concise, Powerful)

COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



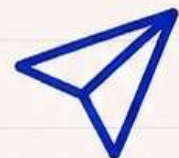
TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.



SAVE



FOLLOW



SHARE

JAVA INTERVIEW IMPORTANT PROGRAMS

Part - 1

Page - 1

1. Java program to Reverse Number

Question : Write a program to reverse a given number.

Algorithm :

- ① Read the number from user.
- ② Initialize rev = 0.
- ③ Extract last digit using num % 10.
- ④ Add digit to rev $\rightarrow$ rev = rev * 10 + r.
- ⑤ Remove last digit $\rightarrow$ num = num / 10.
- ⑥ Repeat steps ③ to ⑤ until num becomes 0.
- ⑦ Print rev.

Code :

```
import java.util.*;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int rev = 0, r;
        while(num != 0) {
            r = num % 10;
            rev = rev * 10 + r;
            num = num / 10;
        }
        System.out.println("Reverse : " + rev);
    }
}
```

Sample Input : 12345

Output : 54321

Time Complexity : $O(d)$ where d = number of digits

Space Complexity : $O(1)$

Note :

- Any number which reads same from left to right and right to left is called Palindrome.
- Example Palindrome numbers : 121, 12321, 7, 1001
- Example non-palindrome numbers : 123, 4567, 789

2. Java program to find Palindrome Number

Question : Write a program to check whether a number is palindrome or not.

Algorithm :

- ① Read the number from user.
- ② Store the number in another variable (originalNumber).
- ③ Reverse the number (same as previous program).
- ④ Compare originalNumber and rev.
- ⑤ If equal $\rightarrow$ palindrome
else $\rightarrow$ not palindrome.

Code :

```
import java.util.*;
public class PalindromeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int originalNumber = num;
        int rev = 0, r;
        while(num != 0) {
            r = num % 10;
            rev = rev * 10 + r;
            num = num / 10;
        }
        if (originalNumber == rev)
            System.out.println("Number is Palindrome");
        else
            System.out.println("Number is not Palindrome");
    }
}
```

Sample Input : 121

Output : Number is Palindrome

Time Complexity : $O(d)$ where d = number of digits

Space Complexity : $O(1)$

3. Java program to find Prime Number

Question : Write a program to check whether a number is prime or not.

Algorithm :

- ① Read the number from user.
- ② If number $\leq 1 \rightarrow$ not prime.
- ③ Check divisibility from 2 to $\sqrt{n}$.
- ④ If number is divisible by any i , then not prime.
- ⑤ If not divisible by any $i \rightarrow$ prime.

Code :

```
import java.util.*;
public class PrimeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        boolean isPrime = true;
        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime)
            System.out.println(n + " is a Prime");
        else
            System.out.println(n + " is not a Prime");
    }
}
```

Sample Input : 17

Output : 17 is a Prime

Time Complexity : $O(\sqrt{n})$

Space Complexity : $O(1)$

Note :

- Prime numbers are numbers greater than 1 which have exactly two factors: 1 and itself.
- Example Prime numbers : 2, 3, 5, 7, 11, 13, 17, ...

4. Java program to find Armstrong Number

Question : Write a program to check whether a number is Armstrong or not.

Algorithm :

- ① Read the number from user.
- ② Count total number of digits in the number.
- ③ For each digit, calculate $(\text{digit}^{\text{count}})$ and add to sum.
- ④ If $\text{sum} == \text{original number} \rightarrow$ Armstrong else $\rightarrow$ Not Armstrong.

Code :

```
import java.util.*;
public class ArmstrongNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int n = sc.nextInt();
        int temp = n, sum = 0, count = 0;
        // Count total number of digits
        while (temp > 0) {
            temp = temp / 10;
            count++;
        }
        // Calculate sum of  $(\text{digit}^{\text{count}})$ 
        temp = n;
        while (temp > 0) {
            int digit = temp % 10;
            sum = sum + (int) Math.pow(digit, count);
            temp = temp / 10;
        }
        if (sum == n)
            System.out.println(n + " is Armstrong number");
        else
            System.out.println(n + " is not Armstrong number");
    }
}
```

Sample Input : 153

Output : 153 is Armstrong number

Time Complexity : $O(d)$ where d = number of digits

Space Complexity : $O(1)$

Note :

- Armstrong numbers are numbers that are equal to the sum of their own digits each raised to the power of total digits.
- Example Armstrong numbers : 0, 1, 153, 370, 371, 407, ...

JAVA INTERVIEW IMPORTANT PROGRAMS

Part - 1

Page - 3

5. Java program to Find Odd or Even number

Question : Write a program to check whether a number is Odd or Even.

Algorithm :

- ① Read the number from user.
- ② If $\text{number} \% 2 == 0 \rightarrow \text{Even}$
- ③ Else $\rightarrow \text{Odd}$.
- ④ Print the result.

Code :

```
import java.util.*;
public class OddEven {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        if (num % 2 == 0)
            System.out.println(num + " is Even");
        else
            System.out.println(num + " is Odd");
    }
}
```

Sample Input : 24

Output : 24 is Even

Time Complexity : $O(1)$

Space Complexity : $O(1)$

Note :

- If number is divisible by 2, it is Even.
- Otherwise, it is Odd.
- 0 is considered as Even.

6. Java program to swap two numbers without using third variable

Question : Write a program to swap two numbers without using third variable.

Algorithm :

- ① Read two numbers a and b.
- ② Use arithmetic operations to swap.
($a = a + b$, $b = a - b$, $a = a - b$)
- ③ Print the swapped numbers.

Code :

```
import java.util.*;
public class SwapNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        a = a + b; // Step 1
        b = a - b; // Step 2
        a = a - b; // Step 3
        System.out.println("After swapping :");
        System.out.println("a = " + a + ", b = " + b);
    }
}
```

Sample Input : a = 10, b = 25

Output : a = 25, b = 10

Time Complexity : $O(1)$

Space Complexity : $O(1)$

Note :

- This method works only for integers.
- For very large numbers, ($a + b$) might exceed the integer range.
- XOR swap can also be used.

JAVA INTERVIEW IMPORTANT PROGRAMS

Part - 1

Page - 4

7. Java program to find Fibonacci series upto a given number of terms

Question : Write a program to print Fibonacci series upto n terms.

Algorithm :

- ① Read number of terms n from user.
- ② Initialize first = 0, second = 1.
- ③ Print first two terms.
- ④ For i = 3 to n
 next = first + second
 Print next
 first = second
 second = next
- ⑤ Repeat step 4 until i > n.

Code :

```
import java.util.*;

public class FibonacciSeries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of terms :");
        int n = sc.nextInt();
        int first = 0, second = 1, next;
        System.out.print(first + " " + second + " ");
        for (int i = 3; i <= n; i++) {
            next = first + second;
            System.out.print(next + " ");
            first = second;
            second = next;
        }
    }
}
```

Sample Input : 7

Output : 0 1 1 2 3 5 8

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Note :

- Fibonacci series starts with 0 and 1.
- Each next term is sum of previous two terms.

8. Java program to find Factorial of a given number

Question : Write a program to find factorial of a given number.

Algorithm :

- ① Read number n from user.
- ② Initialize fact = 1.
- ③ For i = 1 to n
 fact = fact * i
- ④ Print fact.

Code :

```
import java.util.*;

public class FactorialNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number :");
        int n = sc.nextInt();
        long fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial of " + n + " is " + fact);
    }
}
```

Sample Input : 5

Output : Factorial of 5 is 120

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Note :

- Factorial of 0 is 1.
- Factorial grows very fast, use long for bigger numbers.

Practice these programs and try on your own before interviews! 😊

9. Java program to find Sum of digits of a number

Question : Write a program to calculate the sum of digits of a number.

Algorithm :

- ① Read the number from user.
- ② Initialize sum = 0.
- ③ Extract last digit using num % 10.
- ④ Add digit to sum $\rightarrow$ sum = sum + digit.
- ⑤ Remove last digit $\rightarrow$ num = num / 10.
- ⑥ Repeat step 3 to 5 until num becomes 0.
- ⑦ Print sum.

Code :

```
import java.util.*;
public class SumOfDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int sum = 0;
        while (num > 0) {
            int digit = num % 10;
            sum = sum + digit;
            num = num / 10;
        }
        System.out.println("Sum of digits is : " + sum);
    }
}
```

Sample Input : 12345

Output : 15

Time Complexity : $O(d)$ where d = number of digits

Space Complexity : $O(1)$

Note :

Extract last digit using % 10
and reduce number using / 10.

10. Java program to find Number of digits in a number

Question : Write a program to count the number of digits in a given number.

Algorithm :

- ① Read the number from user.
- ② Initialize count = 0.
- ③ Divide the number by 10.
- ④ Increase count by 1.
- ⑤ Repeat step 3 to 4 until num becomes 0.
- ⑥ Print count.

Code :

```
import java.util.*;
public class CountDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any number : ");
        int num = sc.nextInt();
        int count = 0;
        while (num > 0) {
            num = num / 10;
            count++;
        }
        System.out.println("Number of digits is : " + count);
    }
}
```

Sample Input : 987654321

Output : 9

Time Complexity : $O(d)$ where d = number of digits

Space Complexity : $O(1)$

Note :

Every time we divide the number by 10,
one digit is removed.

11. Java program to reverse a string

Question : Write a program to reverse a given string.

Algorithm :

- ① Read the string from user.
- ② Traverse from end to start.
- ③ Append each character to result.
- ④ Print the reversed string.

Code :

```
import java.util.*;
public class ReverseString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        System.out.println("Reversed String : " + rev);
    }
}
```

Sample Input : hello
Output : olleh

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Note :

We are traversing the string once from end to start.

12. Java program to reverse each word of a given string

Question : Write a program to reverse each word of a given string.

Algorithm :

- ① Read the string from user.
- ② Split the string into words.
- ③ Reverse each word individually.
- ④ Print the words in reversed form.

Code :

```
import java.util.*;
public class ReverseEachWord {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String[] words = str.split(" ");
        String result = "";
        for (String word : words) {
            String rev = "";
            for (int i = word.length() - 1; i >= 0; i--) {
                rev = rev + word.charAt(i);
            }
            result = result + rev + " ";
        }
        System.out.println("Result : " + result.trim());
    }
}
```

Sample Input : Java is good
Output : avaJ si doog

Time Complexity : $O(n)$

Space Complexity : $O(n)$

Note :

We split the string into words and reverse each word separately.

JAVA INTERVIEW IMPORTANT PROGRAMS

Part - 1

Page - 7

13. Java program to find String Palindrome

Question : Write a program to check whether a string is palindrome or not.

Algorithm :

- ① Read the string from user.
- ② Compare characters from start and end moving towards center.
- ③ If all characters match $\rightarrow$ palindrome.
- ④ Else $\rightarrow$ not palindrome.

Code :

```
import java.util.*;
public class StringPalindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string : ");
        String str = sc.nextLine();
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        if (str.equalsIgnoreCase(rev))
            System.out.println("String is Palindrome");
        else
            System.out.println("String is not Palindrome");
    }
}
```

Sample Input : madam

Output : String is Palindrome

Time Complexity : $O(n)$

Space Complexity : $O(n)$

Note :

- We can also compare without using extra space by two-pointer approach.
- equalsIgnoreCase() ignores case.

14. Java program to check Two Strings are Anagrams or not

Question : Write a program to check whether two strings are anagrams or not.

Algorithm :

- ① Read two strings from user.
- ② If lengths are different $\rightarrow$ not anagram.
- ③ Sort characters of both strings.
- ④ If sorted strings are equal $\rightarrow$ anagram.
- ⑤ Else $\rightarrow$ not anagram.

Code :

```
import java.util.*;
public class AnagramString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string : ");
        String str1 = sc.nextLine();
        System.out.print("Enter second string : ");
        String str2 = sc.nextLine();
        if (str1.length() != str2.length()) {
            System.out.println("Not Anagrams");
            return;
        }
        char ch1[] = str1.toCharArray();
        char ch2[] = str2.toCharArray();
        Arrays.sort(ch1);
        Arrays.sort(ch2);
        if (Arrays.equals(ch1, ch2))
            System.out.println("Strings are Anagrams");
        else
            System.out.println("Strings are not Anagrams");
    }
}
```

Sample Input : listen silent

Output : Strings are Anagrams

Time Complexity : $O(n \log n)$

Space Complexity : $O(n)$

Note :

- Anagrams must have same length.
- Sorting both strings and comparing is the most common approach.

JAVA INTERVIEW IMPORTANT PROGRAMS

Part - 1

Page - 8

15. Java program to count occurrence of a character in a string

Question : Write a program to count the occurrence of a given character in a string.

Algorithm :

- ① Read string and character from user.
- ② Initialize count = 0.
- ③ Traverse the string.
- ④ If character matches, increment count.
- ⑤ Print count.

Code :

```
import java.util.*;

public class CountChar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string :");
        String str = sc.nextLine();
        System.out.print("Enter a character :");
        char ch = sc.next().charAt(0);
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == ch) {
                count++;
            }
        }
        System.out.println("Occurrence of '" + ch + "'
            + " is : " + count);
    }
}
```

Sample Input : Hello World , o
Output : 2

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Note :

- We traverse the string once.
- Compare each character with given character.

16. Java program to count number of words in a string

Question : Write a program to count the number of words in a string.

Algorithm :

- ① Read string from user.
- ② Trim leading and trailing spaces.
- ③ Split the string by one or more spaces.
- ④ Count the words.
- ⑤ Print count.

Code :

```
import java.util.*;

public class CountWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string :");
        String str = sc.nextLine().trim();
        if (str.length() == 0) {
            System.out.println("No words");
            return;
        }
        String[] words = str.split("\\s+"); // one or
                                           // more spaces

        int count = words.length;
        System.out.println("Number of words : " + count);
    }
}
```

Sample Input : Java is awesome
Output : 3

Time Complexity : $O(n)$

Space Complexity : $O(n)$

Note :

- trim() removes extra spaces at start and end.
- split("\\s+") splits by one or more spaces.

17. Java program to find LCM of two numbers

Question : Write a program to find LCM of two numbers.

Algorithm :

- ① Read two numbers a and b.
- ② Find greater number.
- ③ Initialize lcm = greater.
- ④ While (true)
 if (lcm % a == 0 && lcm % b == 0)
 break;
 else
 lcm++;
- ⑤ Print lcm.

Code :

```
import java.util.*;
public class LCM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        int greater = (a > b) ? a : b;
        int lcm = greater;
        while (true) {
            if (lcm % a == 0 && lcm % b == 0)
                break;
            lcm++;
        }
        System.out.println("LCM is : " + lcm);
    }
}
```

Sample Input : 12 18

Output : 36

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Note :

LCM can also be calculated as
 $LCM(a,b) = (a * b) / GCD(a,b)$
(Efficient approach).

18. Java program to find HCF (GCD) of two numbers

Question : Write a program to find HCF (GCD) of two numbers.

Algorithm :

- ① Read two numbers a and b.
- ② While (b != 0)
 rem = a % b
 a = b
 b = rem
- ③ Print a (which is HCF).

Code :

```
import java.util.*;
public class HCF {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number : ");
        int a = sc.nextInt();
        System.out.print("Enter second number : ");
        int b = sc.nextInt();
        while (b != 0) {
            int rem = a % b;
            a = b;
            b = rem;
        }
        System.out.println("HCF (GCD) is : " + a);
    }
}
```

Sample Input : 48 18

Output : 6

Time Complexity : $O(\log n)$

Space Complexity : $O(1)$

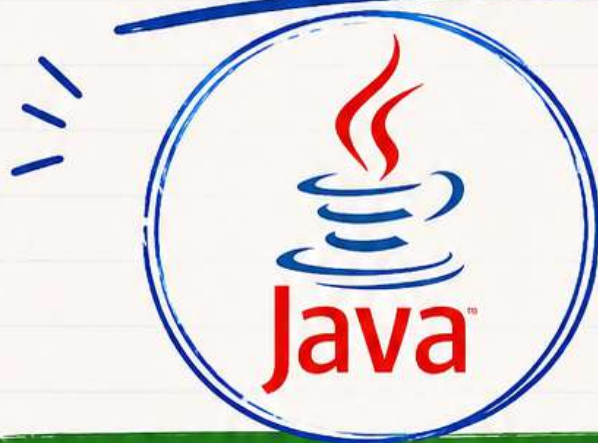
Note :

This is Euclidean Algorithm.
Very efficient for large numbers.

COMMENT PROGRAM FOR

PDF NOTES

FOLLOW FOR PART 2



TARGET 3K LIKES

I WILL SHARE PART 2 



SAVE



SHARE



FOLLOW



Aman Views

